Spatial Modelling (4EK662)
Prostorové modely (4EK462)

Spatial Modelling

Block 1 – Spatial data

Tomáš Formánek

Block 1: Spatial data (TOC)

1. Introduction
2. Vectors/polygons
3. Rasters

Introduction

Introduction

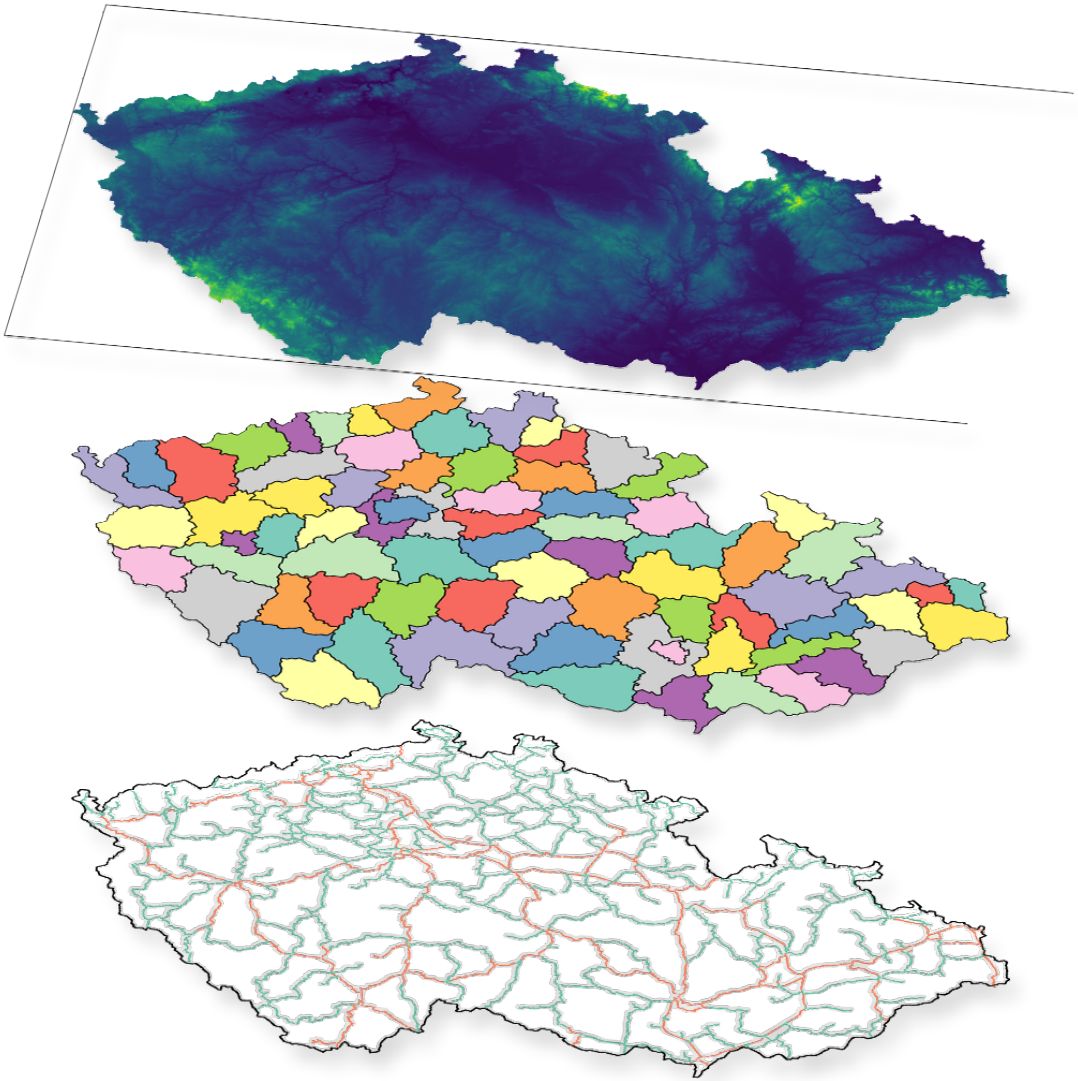
Concepts associated with handling spatial and spatio-temporal data:

- Types of software
  - `R` and other programming languages (Python)
  - GUI (mostly) based software

    `https://www.esri.com/en-us/home` (ArcGIS)

    `https://www.qgis.org/` (QGIS)
- Types of spatial data (Vectors/polygons, rasters, graph-based)
- Projections and coordinate reference systems
- Basic data visualization (maps)

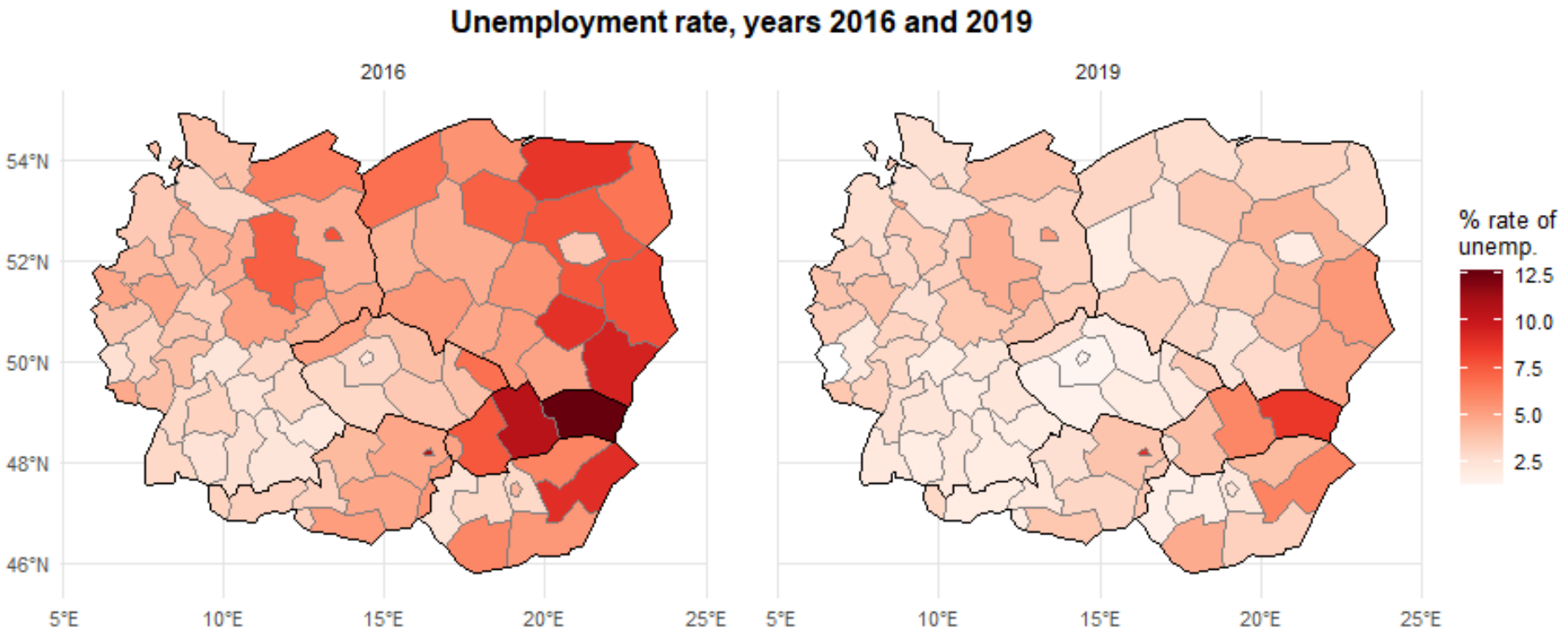
Introduction – Types of spatial data



- Elevation (raster data)
- LAU 1 regions (vector-based)
- Railway network (spatial graph)

Layers share identical geometry (WGS84)

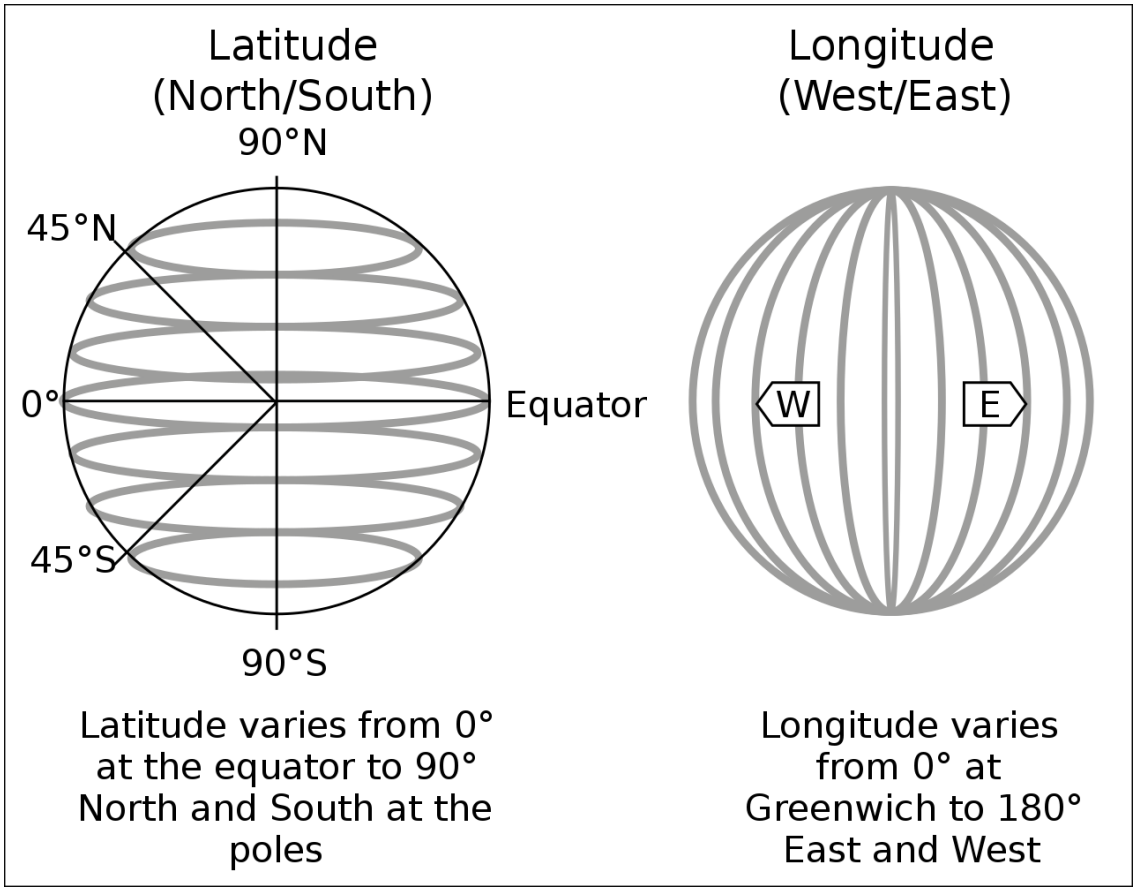
Introduction – Types of spatial data



Spatio-temporal (spatial panel) data.

Introduction – Projections and coordinates

Reference for measuring latitudes and longitudes: Equator and Greenwich

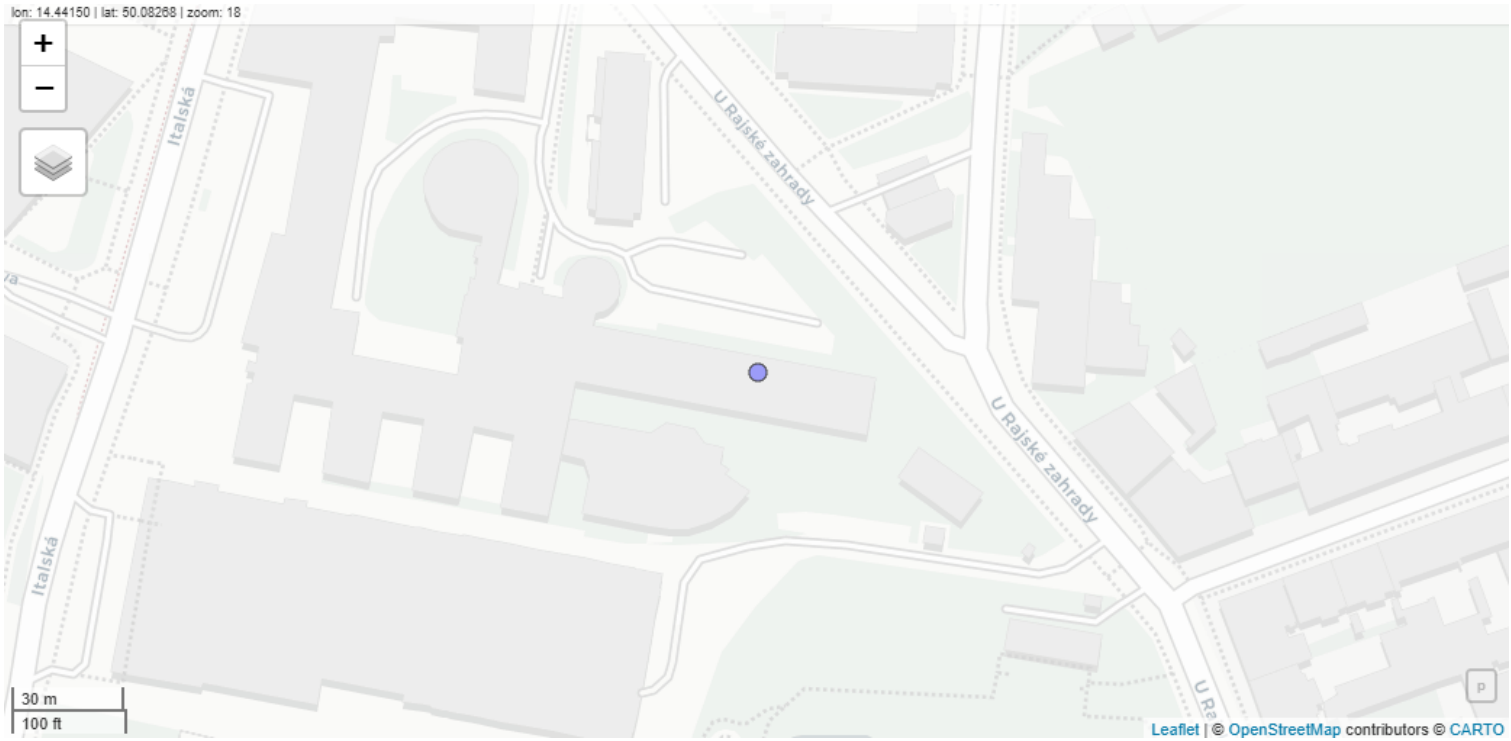


VŠE (W. Churchill sq.): 50.083300N, 14.442408E

Introduction – Creating a Point Geometry in R

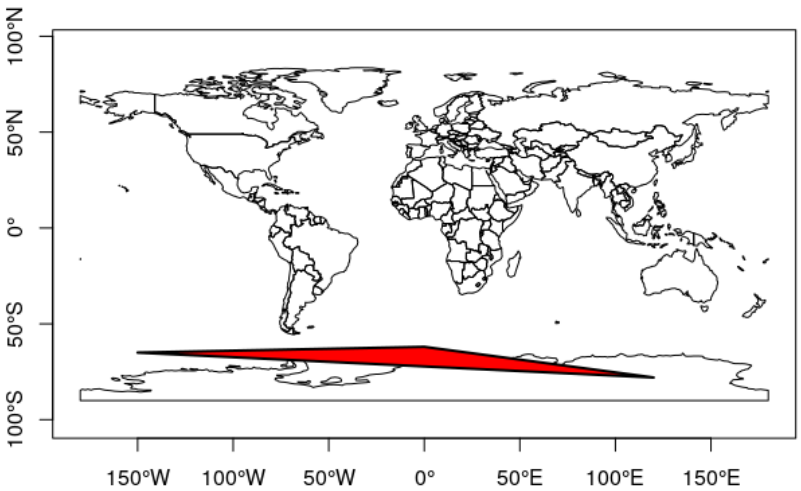
VŠE, W. Churchill sq.: 50.083300N, 14.442408E
(lat-lon × lon-lat)

```
> p <- sf::st_as_sfc("POINT(14.442408 50.083300)",crs = 4326)
# Creates an sfc object
# WKT representation: POINT(lon lat)
# CRS 4326 = WGS84
> p[[1]]
| POINT (14.442408 50.083300)
> mapview::mapview(p)
```

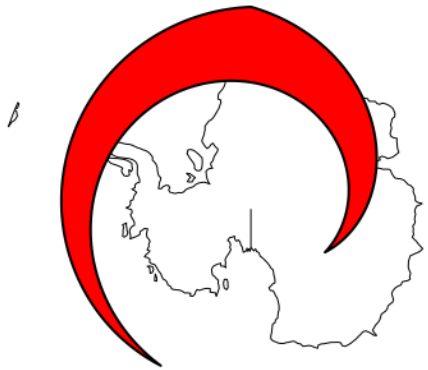


The Earth is not flat . . .

- To plot Earth-based data on a 2D planar surface and to perform certain operations on a planar world, we need to make some (re)projections.
- Depending on where we are, different re-projections of our data might work better than others.

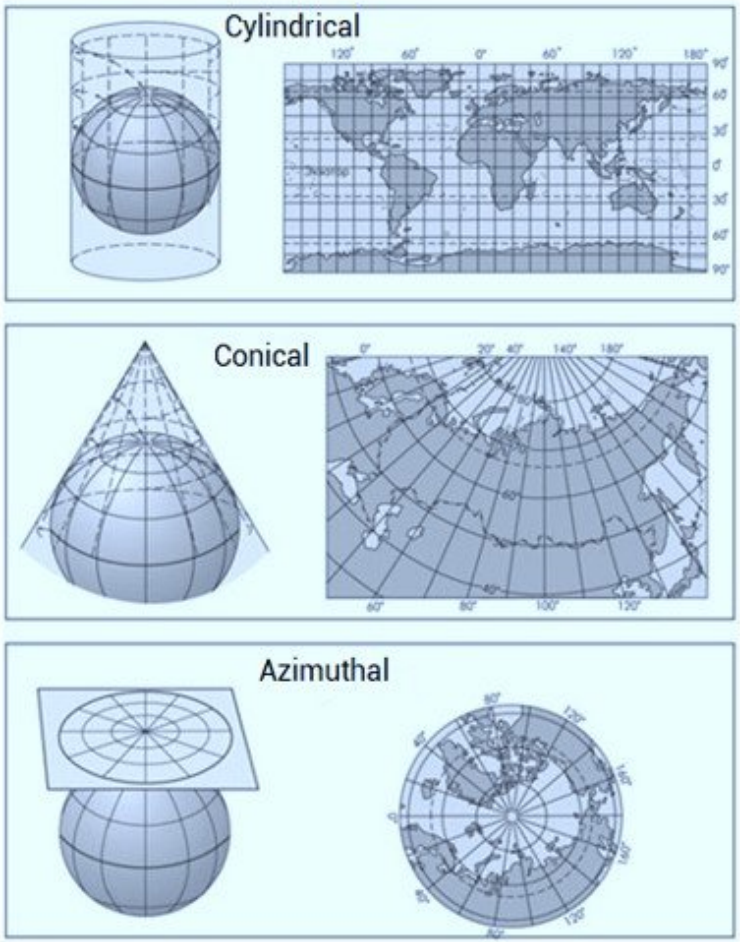


WGS84



Polar projection

Projection examples (three of many)



WGS84 / EPSG:4326

- Basic Coordinate Reference System (CRS) for computerized applications
- Coordinates in degrees, preserves angles, simple
- Major distortion closer to poles

Lambert conic map projection

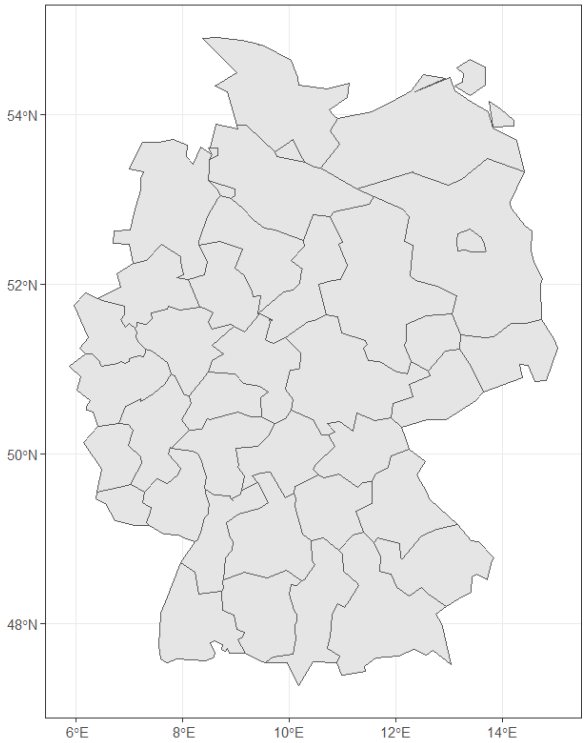
- Directions, angles, and shapes are maintained
- Distances accurate along standard parallels
- Mapping object in an east-to-west orientation rather than north-to-south

Lambert equal area projection

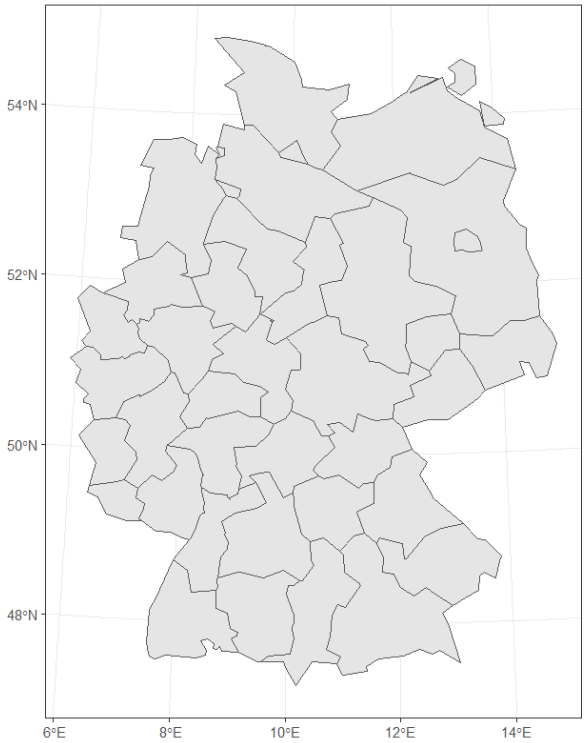
- Shapes, directions, angles, and distances are generally distorted
- Scale and directions are true only at the center of the projection
- Equal-area (equivalent) projection: accurately represents area in all regions

Introduction – Projections and coordinates

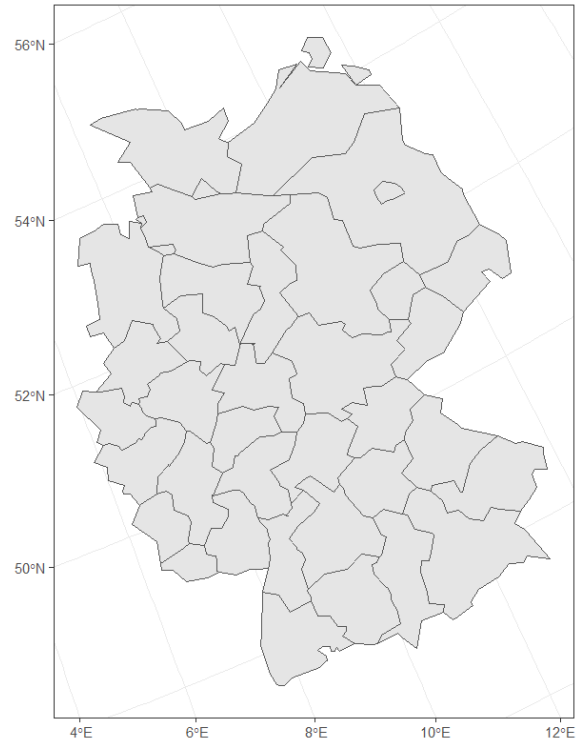
NUTS2-level map of Germany, different projections



WGS84



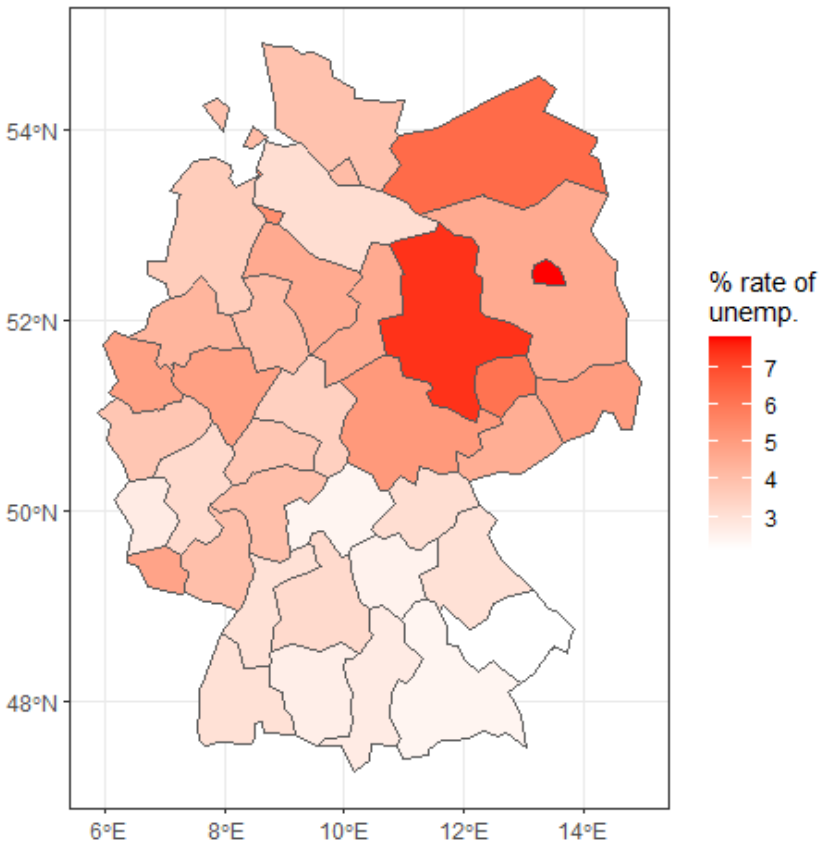
LAEA
(optimized for Germany)



Projection optimized
for Iceland

`ggplot2`-based maps
`geom_sf()` is the key function to handle `sf` objects.

NUTS2-level unemployment, Germany2016 (`sf` and `ggplot2`):



`sf` dataframe `DE`

geo	values	geometry
DE11	3.2	*MULTIPOLYGON (((9.471 49.78...*
DE12	3.0	*MULTIPOLYGON (((9.519 49.47...*
DE13	3.0	*MULTIPOLYGON (((7.96 48.719...*
DE14	2.6	*MULTIPOLYGON (((9.586 48.53...*

`ggplot2` code for the infomap:

```
ggplot(DE) +
  geom_sf(aes(fill = values))+
  scale_fill_gradient(
    name = "% rate of\nunemp.",
    low = "white", high = "red")+
  theme_bw()
```

Note: `geo` column not used for plotting

Introduction – Georeferencing with `tidygeocoder`

```r
addresses <- c(
  "Winston Churchill Square, Prague, Czechia",
  "nám. W. Churchilla, Prague, Czechia",
  "Václavské náměstí, Prague, Czechia",
  "Náměstí Míru, Prague, Czechia")

locs <- tidygeocoder::geo(address = addresses) |>
  as.data.frame()
print(locs)

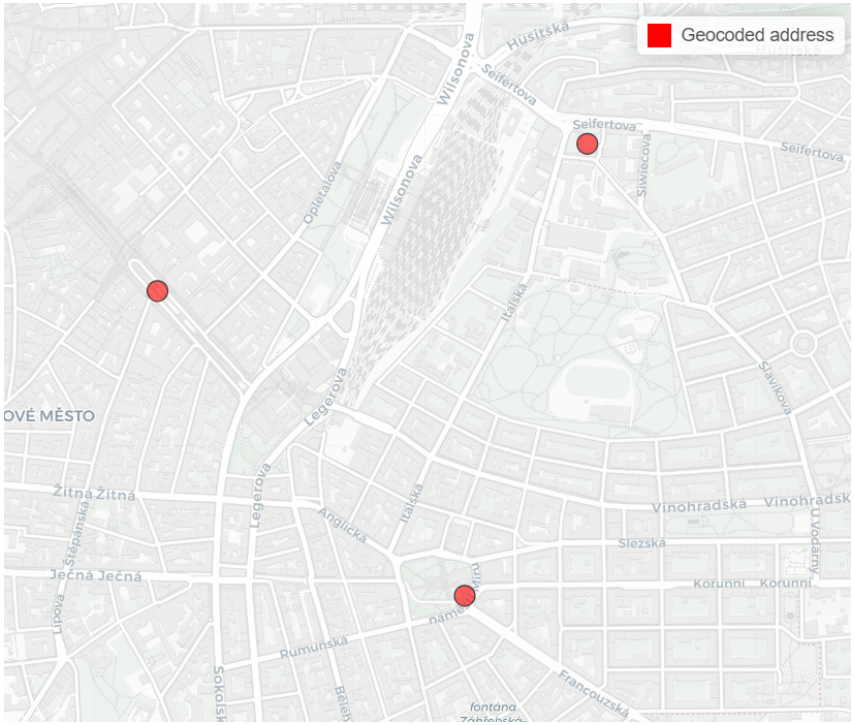
locs_sf <- locs |>
  dplyr::filter(!is.na(long), !is.na(lat)) |>
  # st_as_sf does not work with NAs
  sf::st_as_sf(coords = c("long", "lat"),
               crs = 4326, remove = FALSE)

m <- mapview::mapview(
  locs_sf,
  col.regions = "red",
  cex = 8,
  layer.name = "Geocoded address"
)
m
```

`tidygeocoder::reverse_geocode()`
df with coordinates → query results (addresses)

```r
print(locs)
```

	address	lat	long
1	Winston Churchill Square, Prague, Czechia	NA	NA
2	nám. W. Churchilla, Prague, Czechia	50.08475	14.44142
3	Václavské náměstí, Prague, Czechia	50.08159	14.42717
4	Náměstí Míru, Prague, Czechia	50.07510	14.43737



Introduction - `R` lab

- `R` recap: `.rmd` `.qmd` files, `tidyverse` packages
- `R` packages `sf`, `terra`, (`igraph`, `sfnetworks`)
- Projections and CRS
- Choropleths / infomaps
- Georeferencing

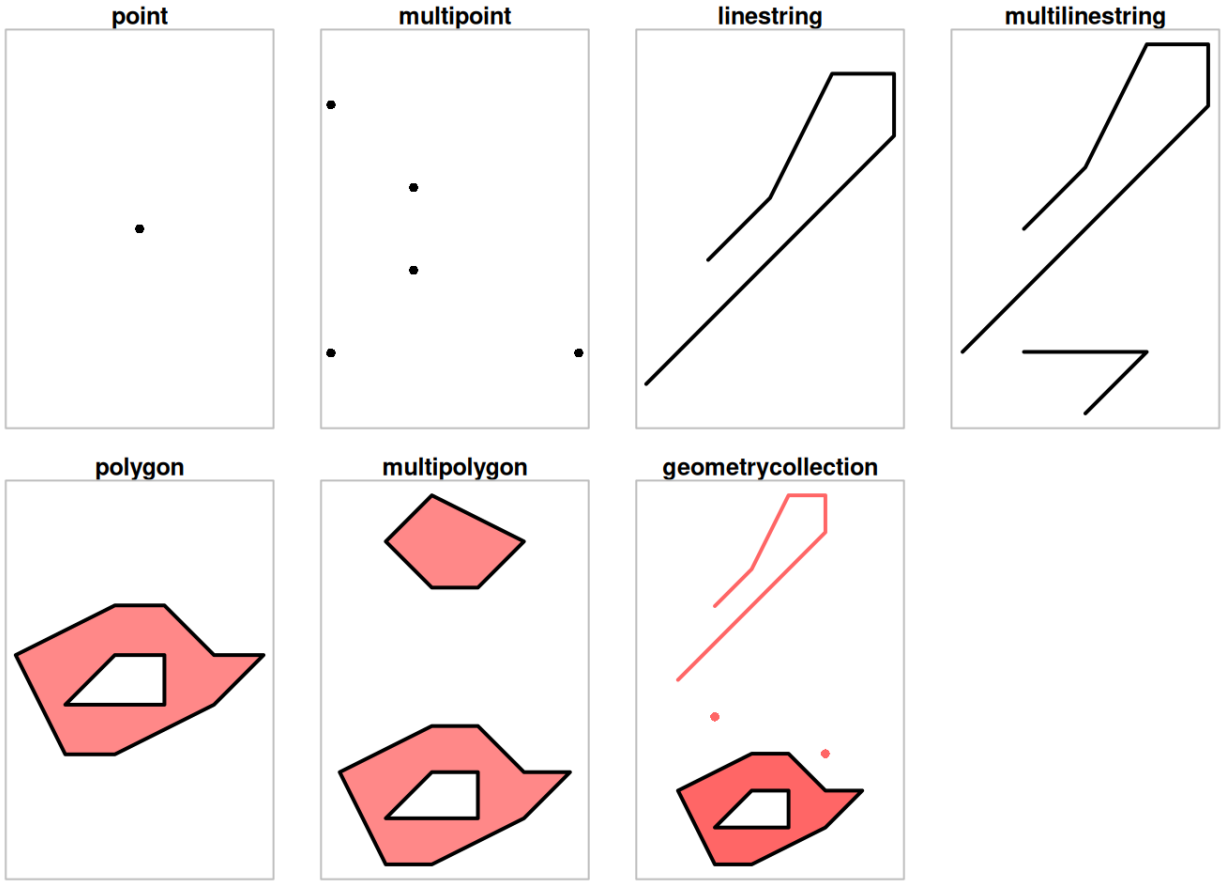
Vectors/polygons

Vectors/polygons

This section focuses on:

- Vector-based geometries
- Spatial objects: `sf` environment
- Data formats (reading and writing spatial files)
- `sf` data operations: logical, measures, transformations
- Map-based calculations and data analysis

Seven basic vector-based geometries (`sfg` objects)



`POINT`	single point geometry
`MULTIPOINT`	set of points
`LINESTRING`	single linestring (2+ points connected by straight lines)
`MULTILINESTRING`	set of linestrings
`POLYGON`	exterior ring with zero or more inner rings (holes)
`MULTIPOLYGON`	set of polygons
`GEOMETRYCOLLECTION`	set of the geometries above

Simple and valid geometries

- `POINT` Individual points in a geometry contain at least two coordinates: x and y. With ellipsoidal coordinates, x and y usually refer to longitude and latitude, respectively.
- `LINESTRING` Linestrings are called simple when they do not self-intersect.
- `POLYGON` Polygon rings are closed (the last point equals the first). Polygon holes (inner rings) are inside their exterior ring, maximally touch the exterior ring in single points, not over a line.
- `MULTIPOLYGON` In a multi-polygon, an external ring maximally touches another exterior ring in single points, not over a line.

Invalid geometries typically cause errors when processed.

Empty geometries

Empty geometries arise naturally from many geometrical operations. For example, when we want to know the intersection of `POINT (0 0)` and `POINT (1 1)`:

```
> e <- st_intersection(st_point(c(0,0)), st_point(c(1,1)))
# GEOMETRYCOLLECTION EMPTY
```

Spatial objects: `sf` package

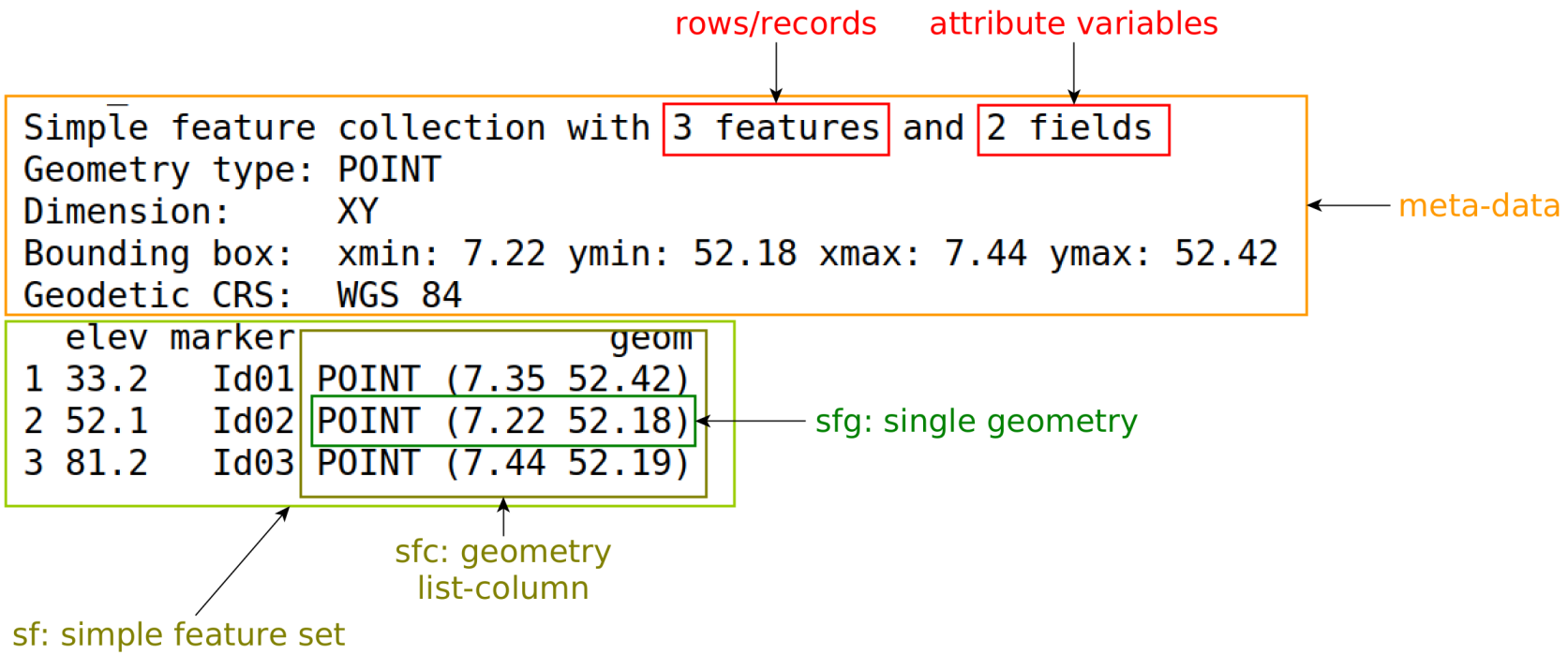
Object hierarchy: `sfg` in `sfc` in `sf`

```
> library(sf)
| Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE
> p1 <- st_point(c(7.35, 52.42))
> p2 <- st_point(c(7.22, 52.18))
> p3 <- st_point(c(7.44, 52.19))
> sfc <- st_sfc(list(p1, p2, p3), crs = 'OGC:CRS84')
> st_sf(elev = c(33.2, 52.1, 81.2),
>       marker = c("Id01", "Id02", "Id03"), geom = sfc)
| Simple feature collection with 3 features and 2 fields
| Geometry type: POINT
| Dimension:     XY
| Bounding box:  xmin: 7.22 ymin: 52.2 xmax: 7.44 ymax: 52.4
| Geodetic CRS:  WGS 84 (CRS84)
|   elev marker              geom
| 1 33.2   Id01 POINT (7.35 52.4)
| 2 52.1   Id02 POINT (7.22 52.2)
| 3 81.2   Id03 POINT (7.44 52.2)
```

- `st_point()` creates an `sfg`: a single spatial geometry element (no CRS)
- `st_sfc()` one or more `sfg` into a `sfc` collection (spatial data only)
- `st_sf()` creates a `sf` object (joins spatial information & data)

Reading sf console output

```
> sfdf <- st_sf(elev = c(33.2, 52.1, 81.2),
>           marker = c("Id01", "Id02", "Id03"), geom = sfc)
> sfdf
```



Spatial objects: `sf` package

Wrangling `sf` objects:

- `st_sfc()` creates `sfc` from `sfg`
- `st_sf()` creates sf, extends data with `sfc`
- `st_as_sf()` converts `df` into `sf`

- `st_drop_geometry()` converts `sf` into `df` (removes `sfc`)
- `st_geometry()` extracts `sfc` from `sf` (removes data)
- `[[]]` list subsetting to extract `sfg` from `sfc`

Note the specific behavior of `sf` objects ("sticky features"): `sfdf2 <- dplyr::select(sfdf,elev,marker)`, or `sfdf2[,1]` does not remove geometry from the target object.

Operations on geometries

Categorization by input

- Single geometry (unary)
- Two geometries (binary)
- Multiple geometries (n-ary)

Categorization by output

- Logical (`TRUE/FALSE`)
- Measures (numeric value, possibly with measurement unit)
- Transformations (newly generated geometries)

Examples of `sf` data operations

Single geometry operations:

- `st_is_valid(x, ...)` checks if (or makes) geometry valid
- `st_area(x, ...)` area of a geometry (polygon/multipolygon)
- `st_bbox(x, ...)` bounding of a geometry (or a set of geometries)

Two-geometry operations:

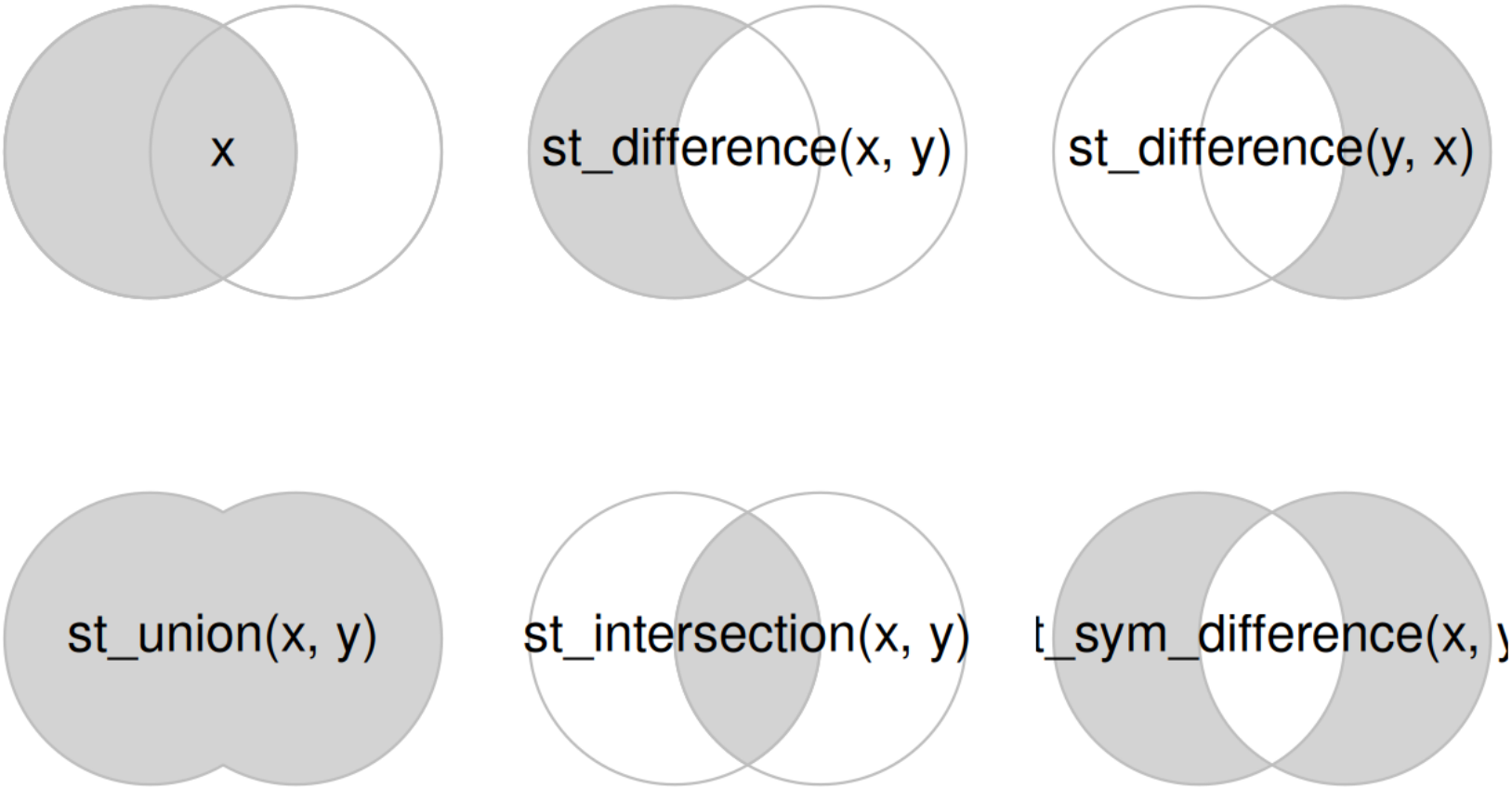
- `st_disjoint(x, y, ...)` `x` and `y` have no points in common
- `st_distance(x, y, ...)` distance between `x` and `y`
- `st_intersection(x,y, ...)` returns the overlapping of two geometries

Multiple geometries (n-ary) operations:

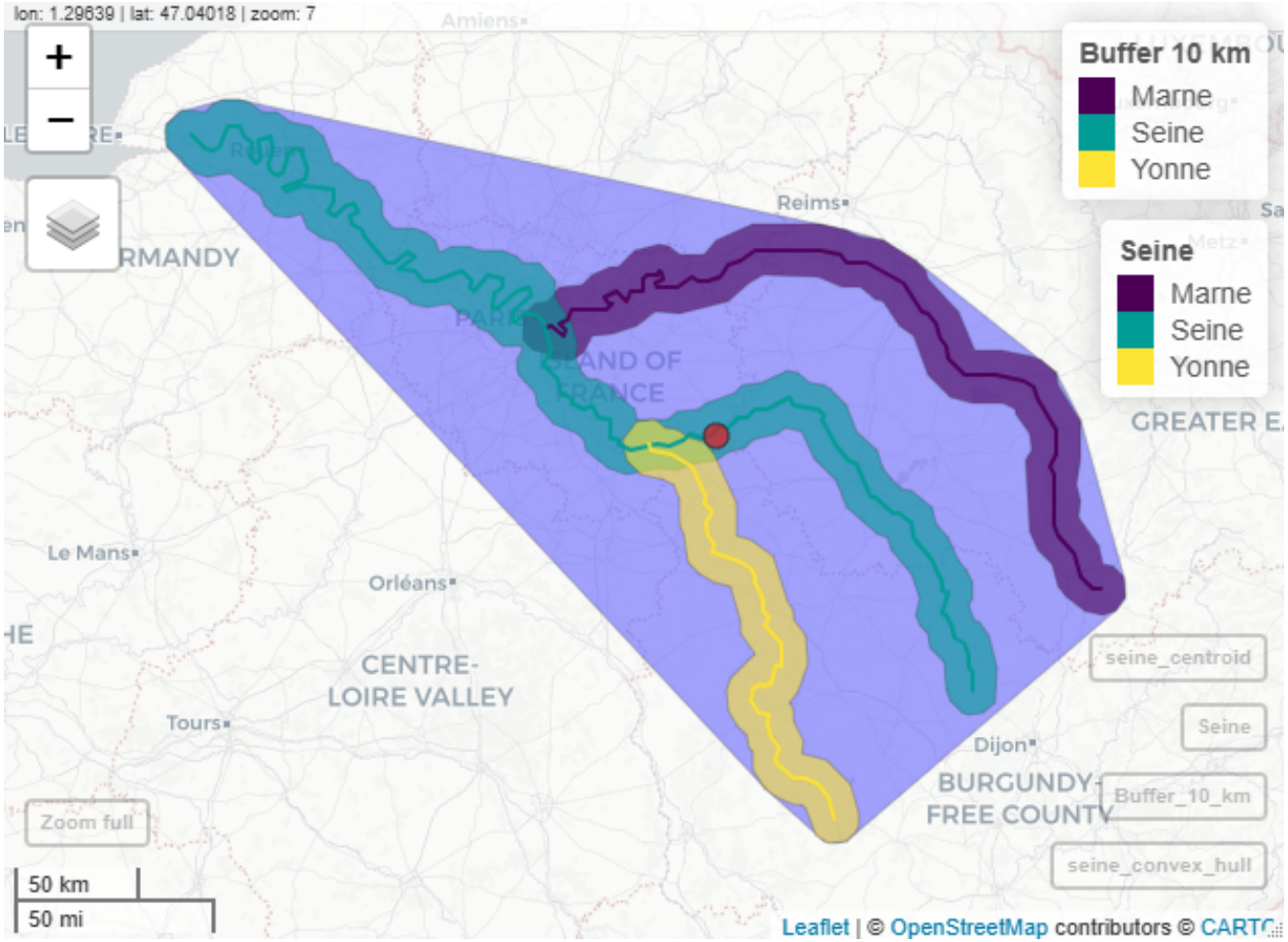
- Often, functions that operate on single/two-geometry objects, can be used on (sequentially applied to) multiple objects (pairs of objects).
- `st_union(obj, ...)` merges geometries, produces one `sfg` (or single-geometry `sfc`).

Examples of `sf` data operations

Spatial filtering (selected `sf` functions):



Examples of `sf` data operations




```
seine_buff_10km <- st_buffer(spData::seine, dist = 10000)
s_c_hull <- seine_buff_10km |> st_union() |> st_convex_hull()
seine_centroid <- st_centroid(seine_c_hull)
```

Map-based calculations and data analysis

Examples of map-based calculations:

- `st_area(x)`
- `st_length(x)`
- `st_distance(x,y, ...)`
  2D distances, using current CRS
- `st_contains(x,y, ...)`
  is `x` within `y` (e.g. point in polygon)?
- `st_is_within_distance(x,y)`
  is `x` within a given distance from `y`?

- Length(s) of common border(s)
- Length(s) of highways (km) across regions
- Sort regions by area
- Higway density (km highway / 100 km^2)
- Number of bus-stops (other OSM-based amenities) within city districts
- Population density, relative area occupied by forests, ...

Vectors/polygons – `R` lab

- Wrangling spatial geometries
- `sf` data operations
- Infomaps (choropleths)
- Map-based calculations

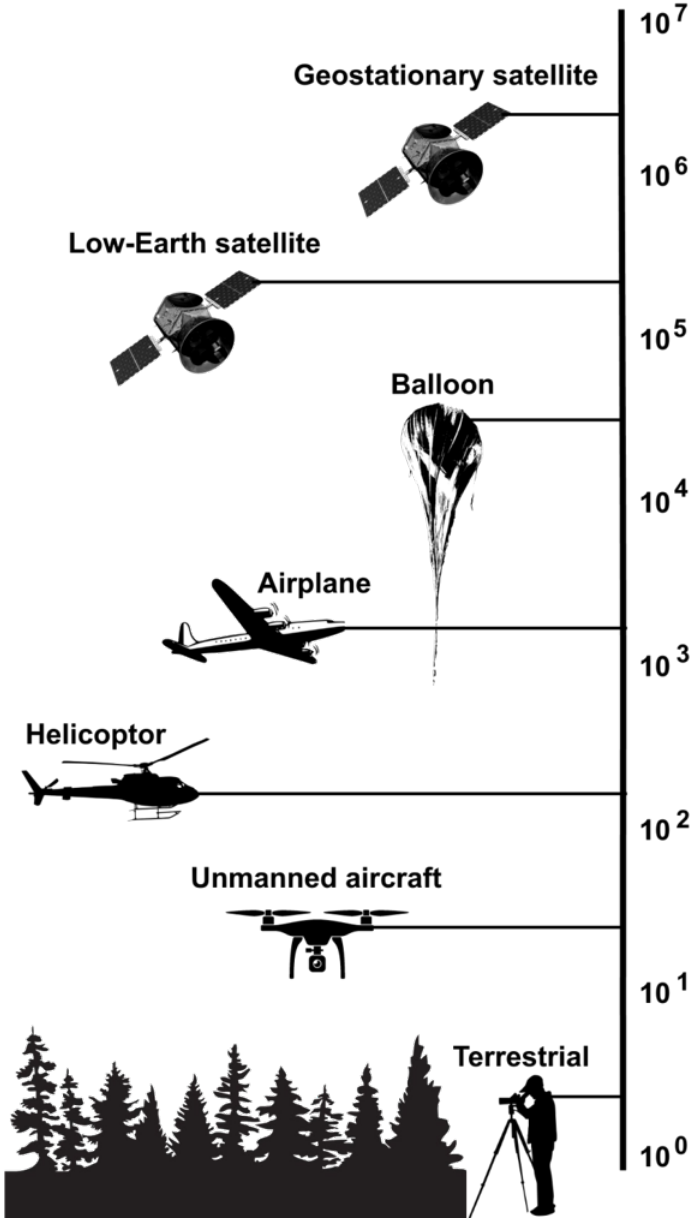
Rasters

Rasters

This section focuses on:

- Remote sensing basic
- Rasters
- `terra` package (`SpatRaster` and `SpatVector` classes)
- Data operations on rasters
- Reading and writing raster data
- Combined data analysis based on rasters and `sf` objects

Remote sensing basics



Remote sensing platforms

The choice of platform can impact everything from the type of imagery that can be collected to the scale and frequency of the imagery.

The higher above Earth the platform is, the more expensive it becomes, but the cost per area imaged is reduced drastically: e.g. handheld camera vs satellite imagery.

(https://www.opengeomatics.ca/)

Remote sensing basics

Remote sensing: any method of gathering information about an object/objects without physical contact.

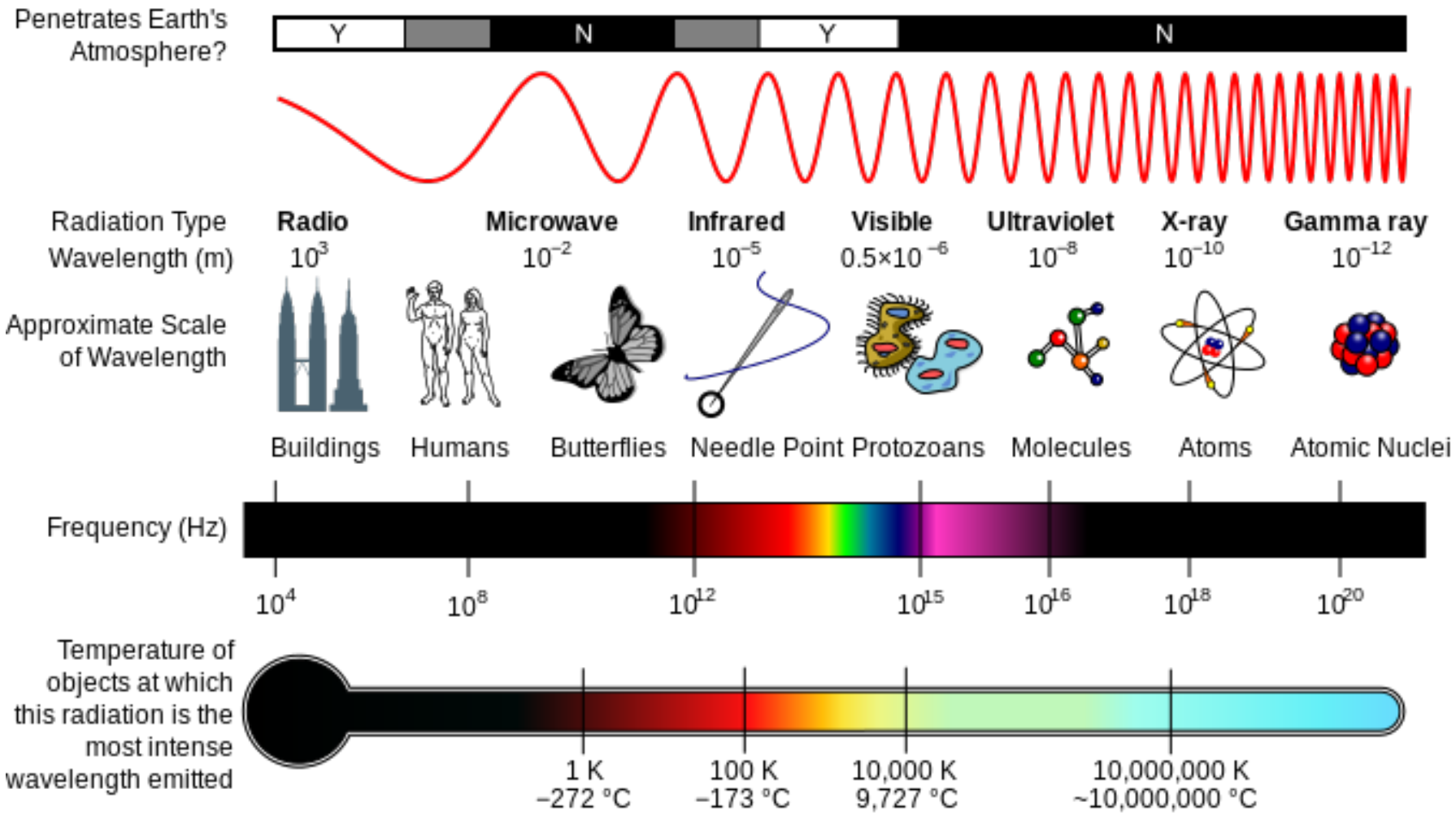
At its core, remote sensing is simply the measurement of **photons**.

Wavelength: a fundamental component of a photon, the measured space between two consecutive peaks of a wave. It determines if and how a photon interacts with the particles around it, & defines the amount of **energy** it has.

$$E = \frac{hc}{\lambda}$$

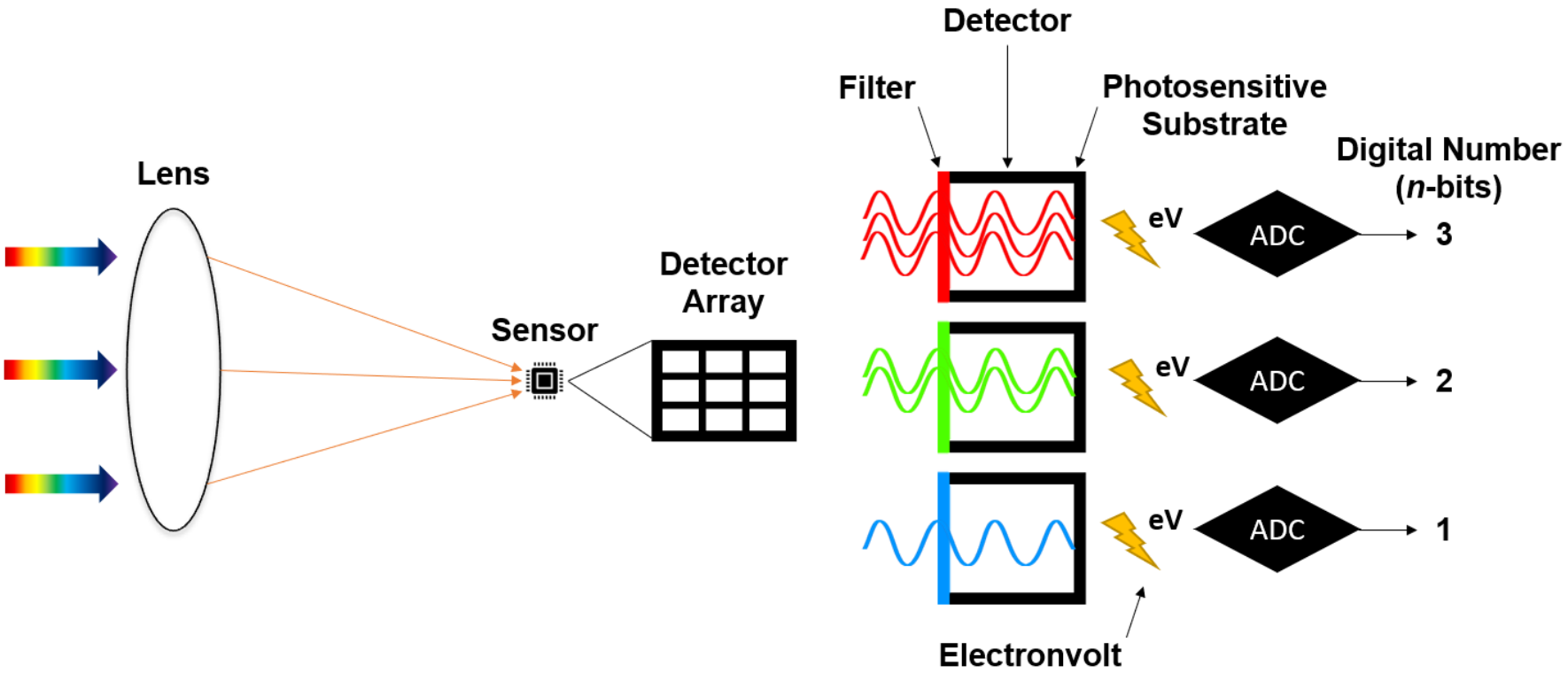
where:
E is the energy of a photon,
h is Planck's constant ($6.62607015 \times 10^{-34} Js$),
c is the speed of light ($c = 3 \times 10^8 m \cdot s^{-1}$) and
λ is the wavelength of the radiation
($hc \approx 1240$ eV$\cdot nm$ is commonly used in photonics, SI: $1.986 \times 10^{-25} J \cdot m$)

Remote sensing – electromagnetic spectrum



Electromagnetic spectrum plot showing the type, wavelength, frequency and black body emission temperature.

Remote sensing basics – sensors



EMR enters the lens, is refracted and focused onto a digital sensor. An array of detectors are arranged on the digital sensor that **represent different pixel locations in the output raster image**. Filters ensure only specific wavelengths are recorded by each detector. Energy from photons is converted to electrical charges and then to numbers by the analog-to-digital converter.

Remote sensing – electromagnetic radiaton

In terms of interaction with other objects, EMR can be partitioned into three normalized components:

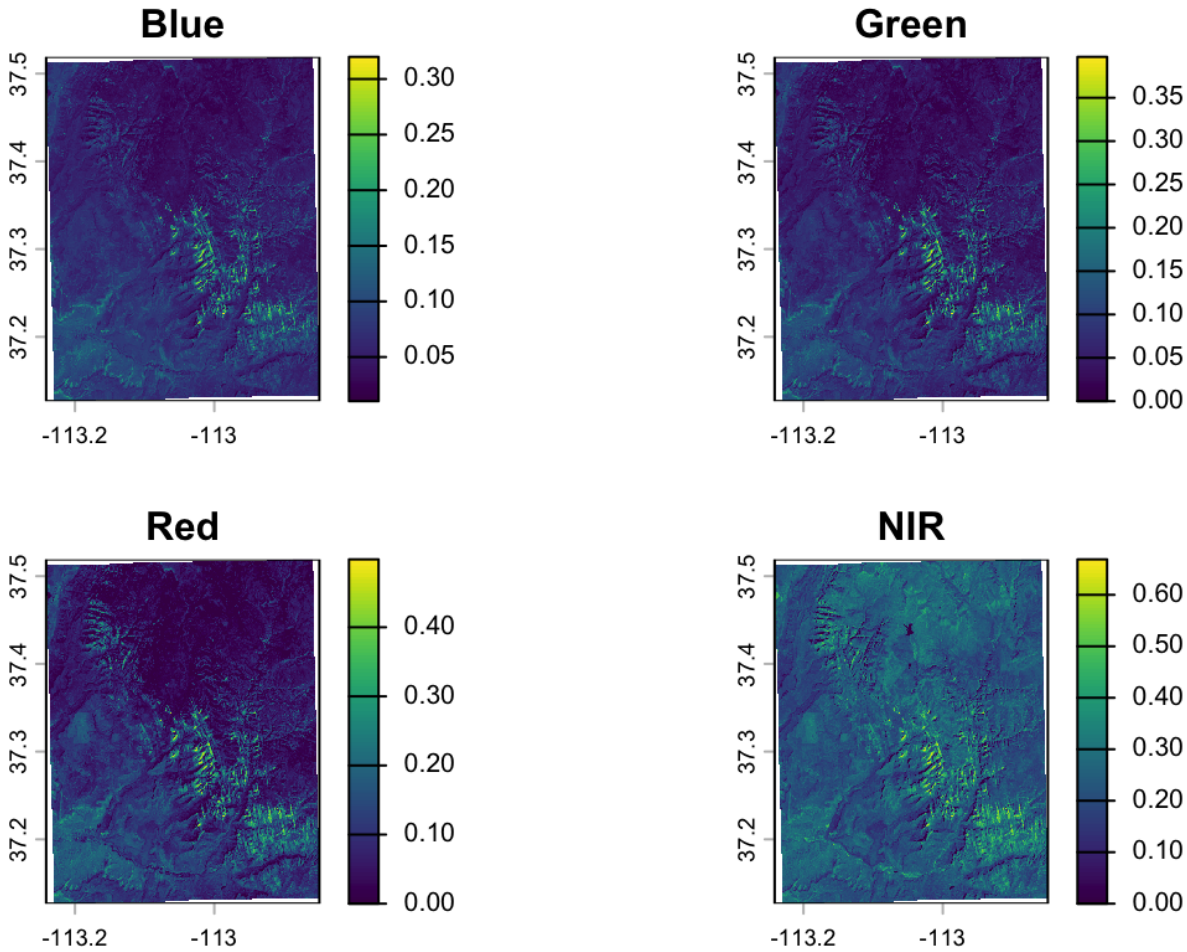
- **Reflectance (R):** energy reflected by the surface. Used for evaluating a variety of environmental properties, including land use change, plant health and plant diversity (NDVI).
- **Transmittance (T):** energy passing through the surface. Indicates how much energy can reach other surfaces below/beyond. For example, it determines the amount of radiation that reaches below the upper canopy of a forest.
- **Absorptance (A):** energy absorbed by the surface. Amount of energy absorbed by (the cells within) a surface; and is roughly equal to the amount of energy not captured as reflectance or transmittance

$$\boxed{R + T + A = 1} \qquad A = 1 - R - T$$

This simple definitions allows for a variety of remote sensing applications. (healthy and stressed vegetation absorbs/reflects Red and NIR differently)

Remote sensing basics – example

Landsat 8: R/G/B/NIR bands, Zion National Park, Utah (USA)

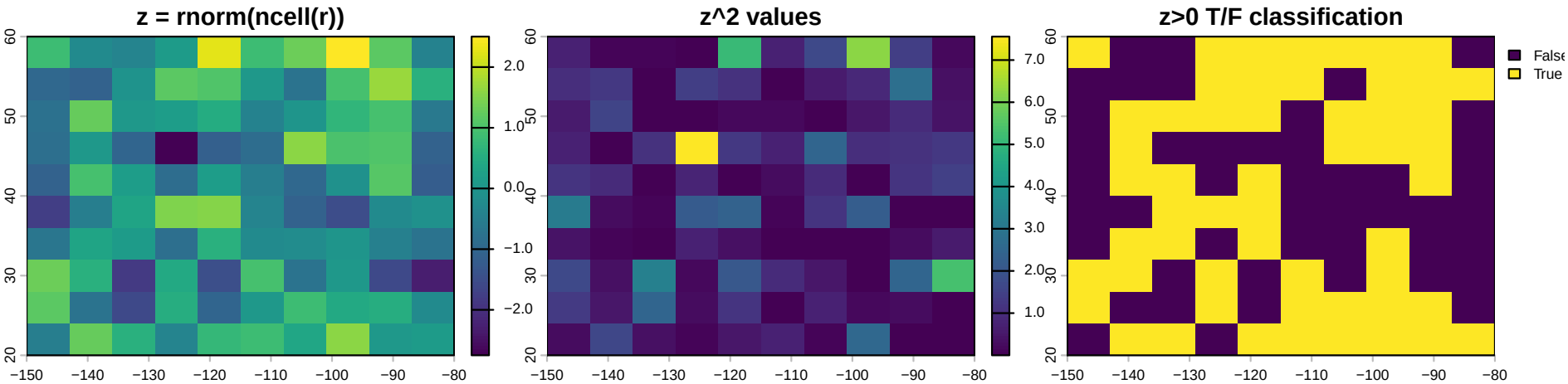


Rasters

- **Raster data:** to represent spatially continuous phenomena (elevation)
- **Raster:** divides the field (world) into a grid of equally sized rectangles: cells (pixels with satellite remote sensing)
- **Cell:** can only store one value in a single layer (unlike `sf` objects).
- **Layers:** used to store multiple values for individual cells.

`raster` object example, 3 layers:

```
size : 10, 10, 3 (nrow, ncol, nlyr)
```



Rasters

- **Raster object:** header + raster matrix
- **Header:** defines CRS, the extent, and origin.
- **Extent:** number of rows, columns, and cell size (resolution)
- **Resolution** $= \left[\frac{x_{\max} - x_{\min}}{\text{ncol}}, \frac{y_{\max} - y_{\min}}{\text{nrow}}\right]$
- **Origin:** `terra` uses the upper-left corner as origin by default

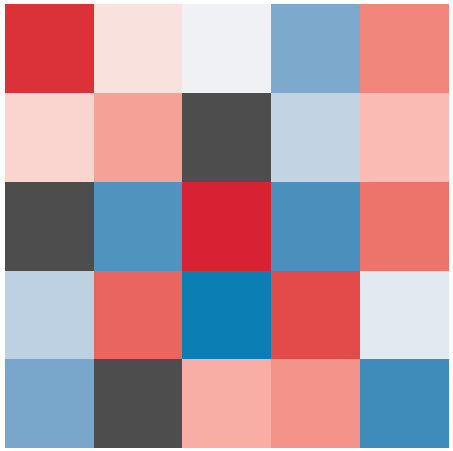
A. Cell IDs

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

B. Cell values

92	55	48	21	76
58	70	NA	37	64
NA	12	94	11	80
36	83	4	88	45
20	NA	67	73	9

C. Data visualization



Rasters – retrieving cell coordinates and ID example

- Raster geometry is implicitly set by knowing the spatial extent and the number or rows and columns in which the area is divided.
- Raster geometry not stored as coordinates (unlike `sf` objects).
- Raster cells can be handled as a set of regular polygons – yet it is inefficient to represent the data that way (coordinates for each cell would have to be stored explicitly, increasing size and processing time).
- `terra`: Cell ID internally, $[x, y]$ coordinates of a cell can be produced
- `terra` has a set of functions to retrieve cell / column / row IDs and coordinates: `xyFromCell()` $\leftrightarrow$ `cellFromXY()`.

Rasters – geometry and cell ID

```
> r <- rast(nrows = 5, ncols = 5, xmin = 0, xmax = 5, ymin = 0, ymax = 5)
> r
| class       : SpatRaster
| size        : 5, 5, 1  (nrow, ncol, nlyr)
| resolution  : 1, 1  (x, y)
| extent      : 0, 5, 0, 5  (xmin, xmax, ymin, ymax)
| coord. ref. : lon/lat WGS 84 (CRS84) (OGC:CRS84)
> xyFromCell(r, 1)
|         x   y
| [1,] 0.5 4.5
> xyFromCell(r, 13)
|         x   y
| [1,] 2.5 2.5
> cellFromXY(r, cbind(x = 2.5, y = 2.5))
| [1] 13
```

A. Cell IDs

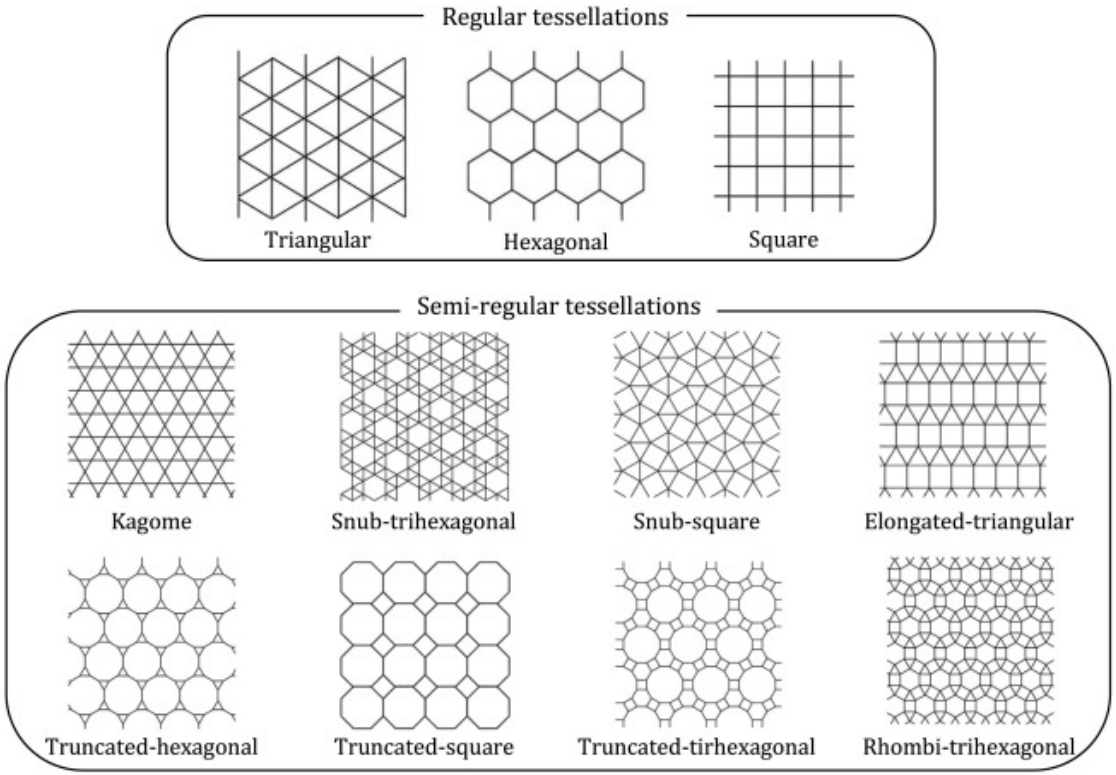
1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

B. Cell−centre coordinates

(0.5, 4.5)	(1.5, 4.5)	(2.5, 4.5)	(3.5, 4.5)	(4.5, 4.5)
(0.5, 3.5)	(1.5, 3.5)	(2.5, 3.5)	(3.5, 3.5)	(4.5, 3.5)
(0.5, 2.5)	(1.5, 2.5)	(2.5, 2.5)	(3.5, 2.5)	(4.5, 2.5)
(0.5, 1.5)	(1.5, 1.5)	(2.5, 1.5)	(3.5, 1.5)	(4.5, 1.5)
(0.5, 0.5)	(1.5, 0.5)	(2.5, 0.5)	(3.5, 0.5)	(4.5, 0.5)

Rasters × hexagons and other polygon tessellations

In `terra`, a raster divides space into a regular matrix of cells: rectangular or square pixels. Other tessellations can be useful in multiple applications, yet they are not objects of the `SpatRaster` class.



Rasters × hexagons and other polygon tessellations

- **Raster** divides space into a regular matrix of cells: rectangular or square pixels (arranged to rows and columns).
  `terra::rast()` → `SpatRaster` class.

- **Polygon tessellation:** divides space into polygons that cover the area without gaps or overlaps.

  Squares, rectangles, hexagons, triangles, Voronoi cells, etc.

  `terra::tessellate()` → `SpatVector` of polygons, not a raster.

- Each polygon has explicit polygon geometry (possibly different shapes/rotations), and there is often an attribute element/row (multiple values can be stored as vector of attributes)

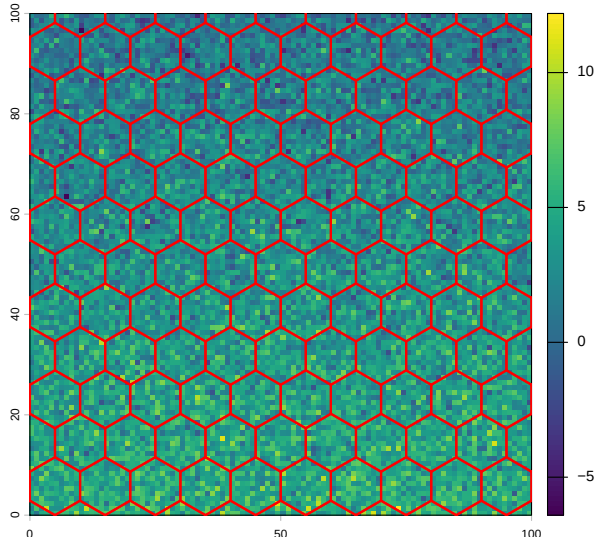
- Often used to aggregate data into spatial units.

Rasters × hexagons and other polygon tessellations

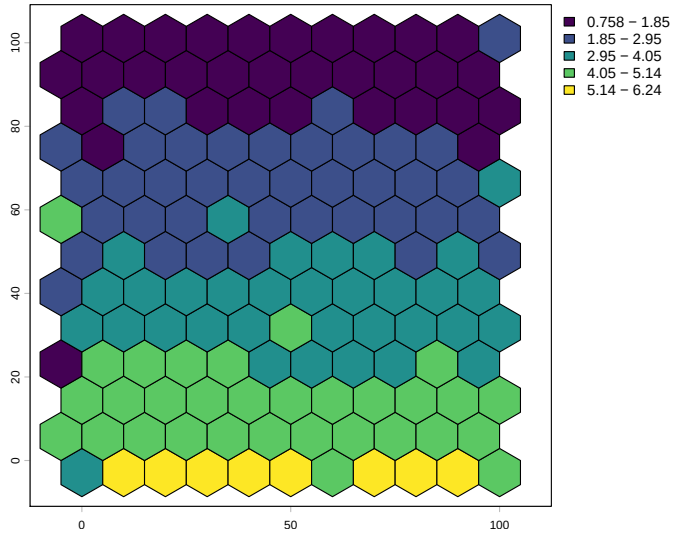
```
> r <- rast(nrows = 100,ncols = 100,
        xmin = 0, xmax = 100,ymin = 0, ymax = 100,
        crs = "local")
| class       : SpatRaster
| size        : 100,100,1  (nrow,ncol,nlyr)
| resolution  : 1,1  (x,y)
| extent      : 0,100,0,100  (xmin,xmax,ymin,ymax)
| coord. ref. : Cartesian (Meter)

> hex <- tessellate(r,size = 10,type = "hexagon")
> hex_mean <- extract(r,hex,fun = mean,na.rm = TRUE)
> hex_min <- extract(r,hex,fun = min,na.rm = TRUE)
> hex$mean_value <- hex_mean[, 2]
> hex$min_value <- hex_min[, 2]
> hex
| class       : SpatVector
| geometry    : polygons
| dimensions  : 149,2  (geometries,attributes)
| extent      : -10,110,-8.660254,106.8098
|               (xmin,xmax,ymin,ymax)
| coord. ref. : Cartesian (Meter)
| names       : mean_value min_value
| type        :      <num>     <num>
| values      :    3.90525  0.877435
|                  6.03817   3.81554
|                  5.15978   1.17613
```

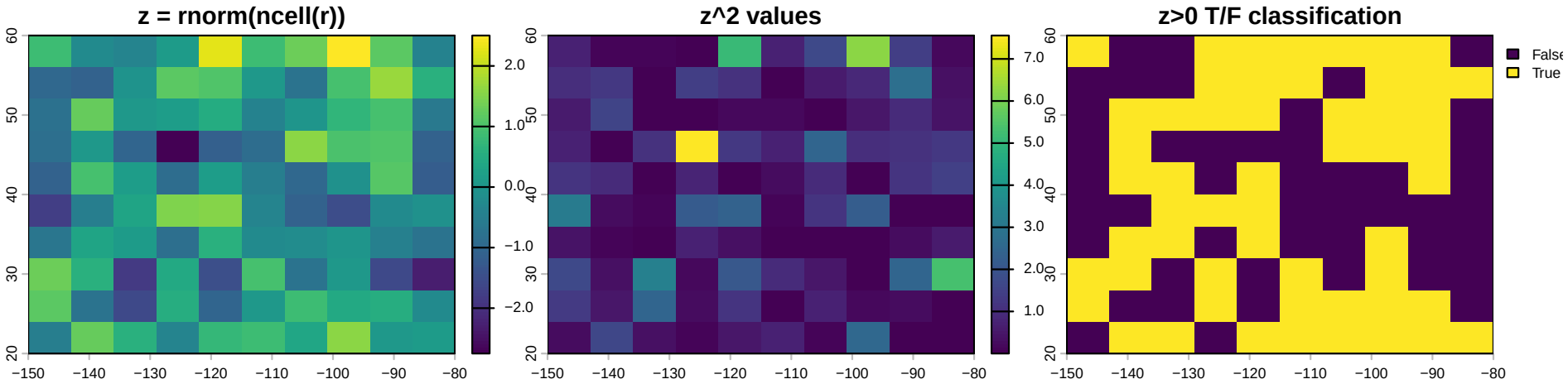
Raster with hexagonal overlay



Mean raster value by hexagon



Raster operations – subsetting

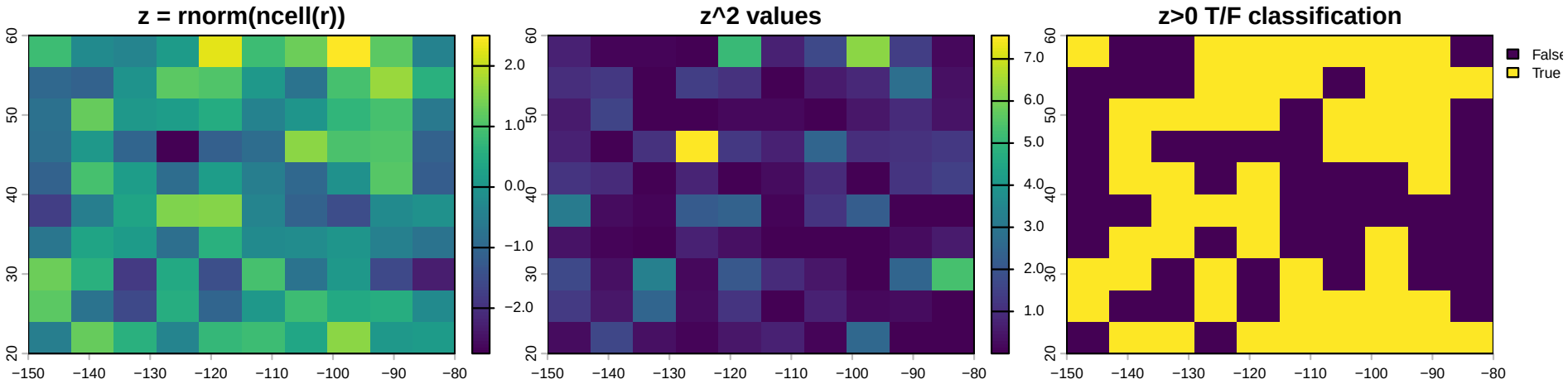



```
> r <- rast(ncol=10, nrow=10, xmin=-150, xmax=-80, ymin=20, ymax=60) # (10 x 10) matrix
> values(r) <- rnorm(ncell(r))
> r2 <- r * r
> r3  <- r > 0
> s <- c(r, r2, r3) # multi-layered raster (3 layers), (10 x 10 x 3) array
> names(s)[c(1:3)] <- c("z = rnorm(ncell(r))", "z^2 values","z>0 T/F classification")
```

Subsetting is used to read or set cell values in a raster / raster layer

```
> r[1] # cell ID, note that layer is not named
|      lyr.1
|  0.8337332
>r[1,1] # row 1, column 1
|      lyr.1
|  0.8337332
> s[1,1,1] # first element of the 3-layer raster (array)
| z = rnorm(ncell(r))
|           0.8337332
```

Raster operations – subsetting




```
# simplified version of s[1,1,]
> s[1,1]
|   z = rnorm(ncell(r)) z^2 values z>0 T/F classification
| 1           0.8337332   0.695111                    TRUE
 # first "row" of s (10 x 3 matrix)
> s[1,]
|    z = rnorm(ncell(r))  z^2 values z>0 T/F classification
| 1           0.83373317 0.695111000                   TRUE
| 2          -0.27604777 0.076202373                  FALSE
| 3          -0.35500184 0.126026305                  FALSE
| 4           0.08748742 0.007654049                   TRUE
| 5           2.25225573 5.072655876                   TRUE
| 6           0.83446013 0.696323707                   TRUE
| ...
> s[]
# same as values(s) - returns all values of cells in s

> s[1,1,1] <- NA # sets the first element to "NA"
> s[1,1] <- c(0.5,0.25,TRUE)
```

Raster operations – spatial subsetting

Besides cell ID or row/column identification, raster elements can be extracted by location: **coordinates** or **other spatial object**.

- Coordinates-based subsetting

```
# For a single raster
# .. note how the coordinates are formatted as a (1x2) matrix
> id <- cellFromXY(r, matrix(c(-118.5,54), ncol =2))
> id
[1]   15
> r[id]
|     lyr.1
| 1 1.053219
# set new value
> r[id] <- 1

# Raster with multiple layers (array): same cell geometry in all layers
# Layer 3 used, same result for any layer. Note the simpler coordinates notation:
> id <- cellFromXY(s[[3]], cbind(-83.5, 42))
> id
[1]  50 # cell No. 50

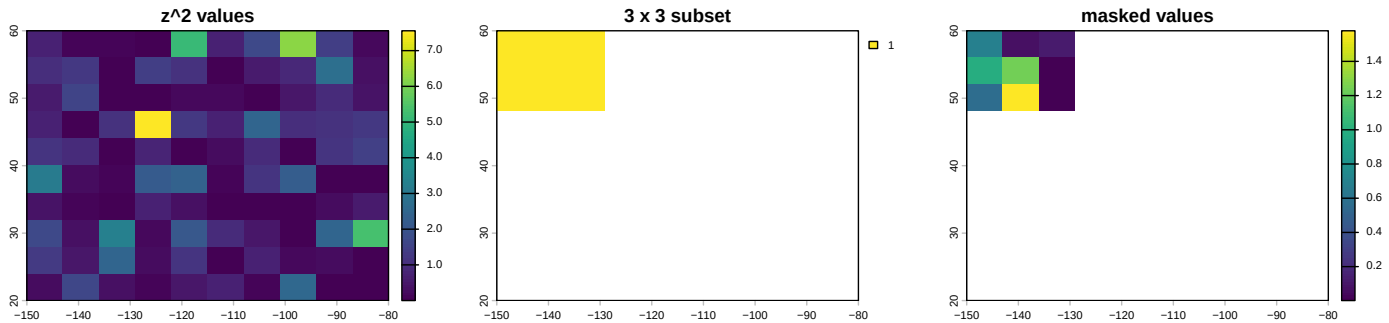
# For subsetting, note the separate layer and XY identification:
> s[[3]][50]
|   z>0 T/F classification
| 1                  FALSE

# The above commands retrieve values from cells. For spatial output, use:
> r_subset <- r[1:20, drop = FALSE]
# Returns raster made of the first (top) two rows of r (cells 1 to 20)
```

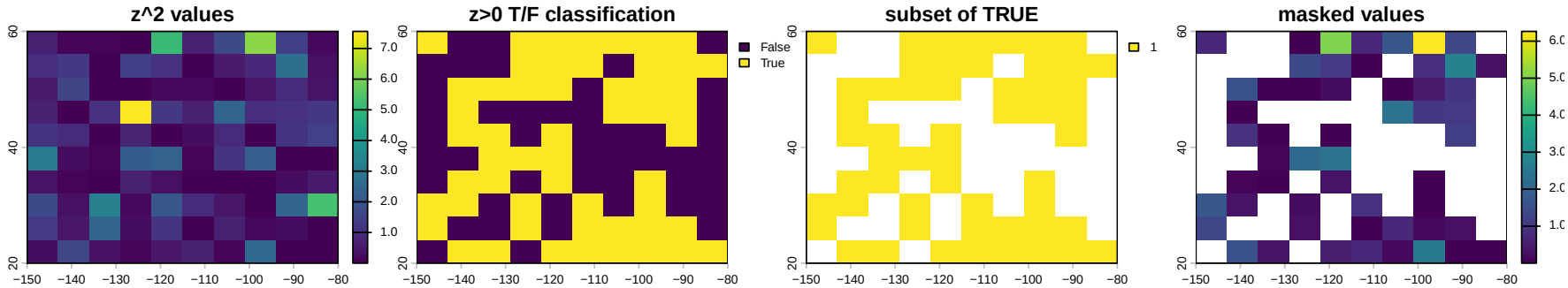
Raster operations – spatial subsetting

- Spatial object-based subsetting

```
# 3x3 subset of raster r2 (10x10); note the "free scales" on plotted rasters/subsetsx
> r3x3 <- r2
> terra::values(r3x3) <- NA
> r3x3[c(1:3,11:13,21:23)] <- TRUE
> r2_3x3 <- terra::mask(r2,r3x3)
# mask() is often used with the crop() function, as shown next
```

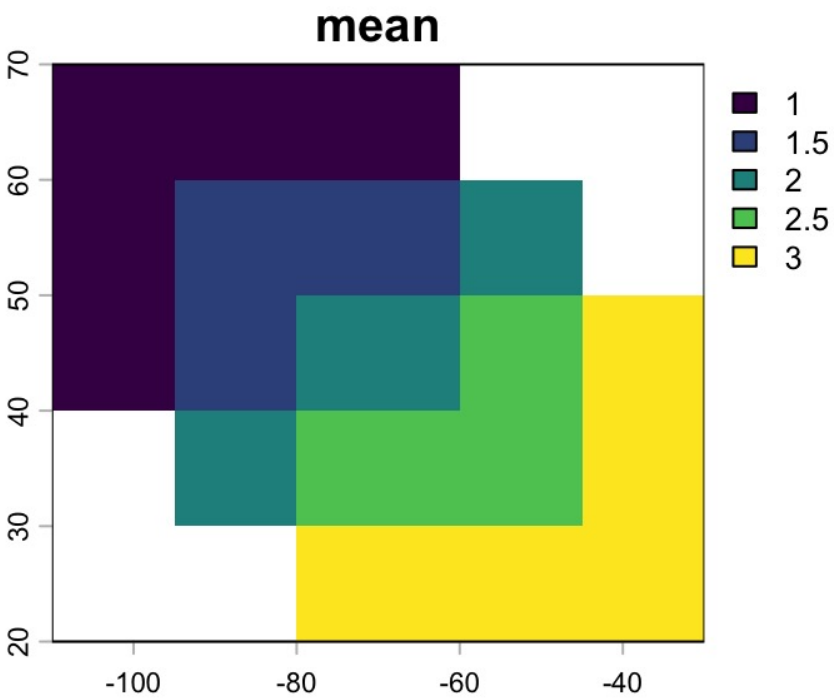


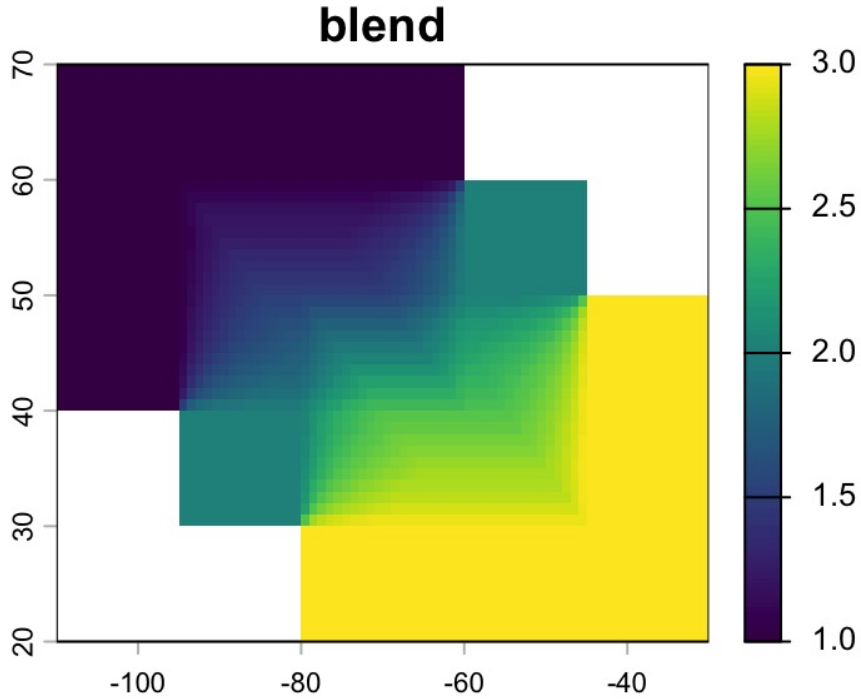

```
# Spatial subset based on TRUE/FALSE evaluation (NA values used for masking)
# terra::ifel(test, yes, no, ...) works on rasters with logical values:
> r3subset <- terra::ifel(r3, TRUE, NA)
> r2_spatial_subset <- terra::mask(r2,r3subset)
```



Raster operations – combining rasters

```
> x <- rast(xmin=-110, xmax=-60, ymin=40, ymax=70, res=1, vals=1)
> y <- rast(xmin=-95, xmax=-45, ymax=60, ymin=30, res=1, vals=2)
> z <- rast(xmin=-80, xmax=-30, ymax=50, ymin=20, res=1, vals=3)
# x,y,z are 3 different rasters, partly overlapping,
# observed values are also different (1,2,3)
> m1 <- mosaic(x, y, z, fun="mean") # fun="mean" is the default
> m2 <- mosaic(x, y, z, fun="blend") # fun="blend" is the smoother
> m3 <- c(m1,m2)
```





Raster algebra

Raster algebra: cartographic calculations (modelling) with raster data:

- **Local:** per-cell calculations. Includes operations on different layers at the same XY coordinates.
  For example: `terra::mosaic(..., fun="mean")`
- **Focal:** neighborhood operations, typically based on a 3×3 grid.
  `terra::focal()`
- **Zonal:** similar to focal operations, but with irregular sizes or shapes of the grids used for calculations.
  `terra::mosaic(..., fun="blend")`, `terra::zonal()`
- **Global:** per-raster operations; returned values (summary functions / output cells) are based on the entire raster
  `terra::distance()`

Raster algebra – Local operations example

NDVI: normalized difference vegetation index

$$NDVI = \frac{NIR - Red}{NIR + Red}, \qquad NDVI \in \langle -1, 1 \rangle,$$

NIR is near-infrared reflectance, at spectral range 760-900 nm (approx.)
Red is red-band reflectance, at 620-700 nm .

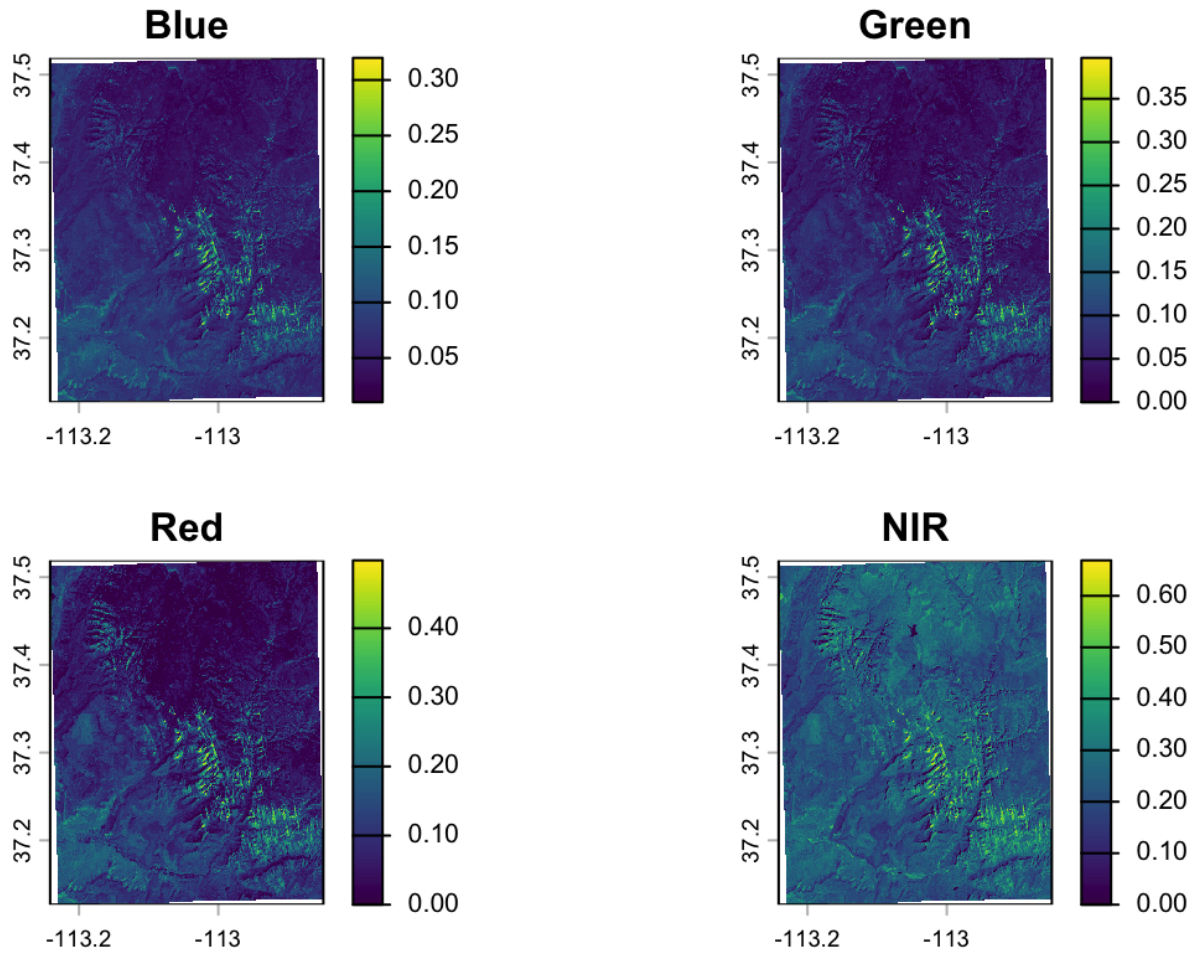
NDVI is used to measure the presence, density, and approximate condition of green vegetation from satellite or aerial imagery.

- healthy vegetation absorbs red light strongly because of chlorophyll
- healthy vegetation reflects near-infrared light strongly (leaf structure)

Vegetation / surface condition	Expected NDVI
Water, clouds, snow, shadows	negative or near 0
Bare soil, built-up land	around 0
Sparse or stressed vegetation	low positive, 0.1–0.3
Moderate vegetation	around 0.3–0.5
Dense, healthy vegetation	high values, 0.5–0.8 or more

Raster algebra – Local operations example

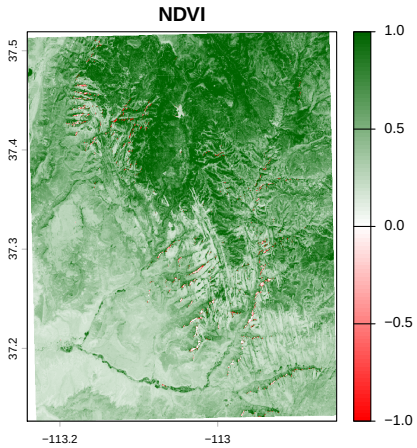
NDVI Landsat 8 example (Red band: 640-670 nm, NIR band: 850-880 nm)
Zion National Park, Utah (USA)

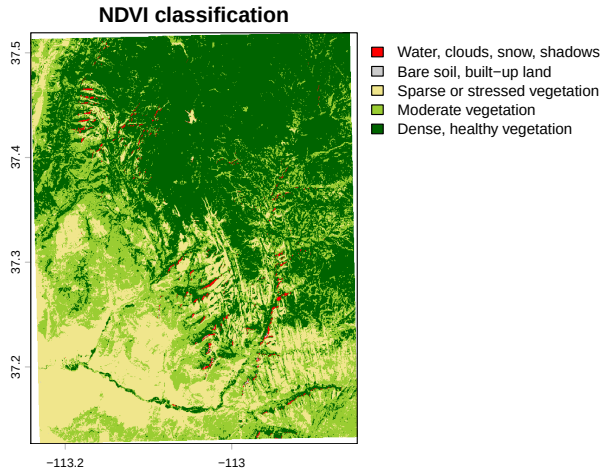


Raster algebra – Local operations example

NDVI Landsat 8 example (Red band: 640-670 nm, NIR band: 850-880 nm)
Zion National Park, Utah (USA)







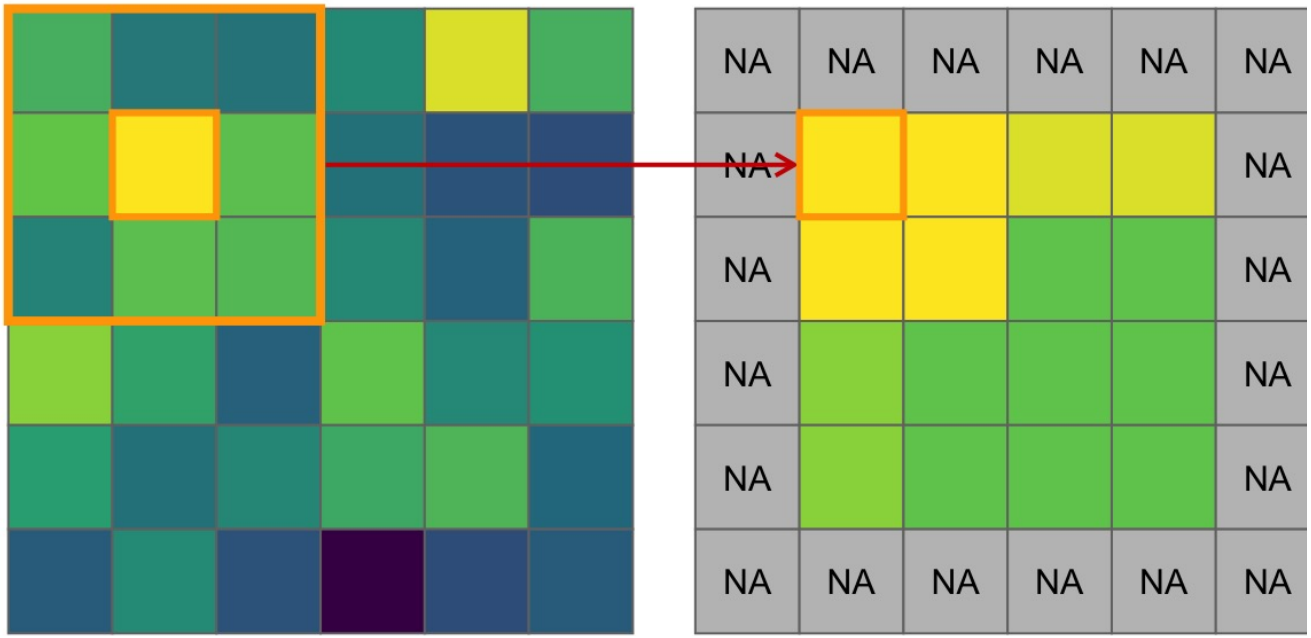
Droughts would lower NDVI as vegetation becomes less green and less dense.
NDVI measures plant response rather than drought directly.

Raster algebra – Focal operations

Focal operations: work with a central (focal) cell and its neighbors.

- 3×3 kernel/filter is typical, variations possible (incl. non-rectangular)
- `terra::focal()`

```
> vals <- matrix(rnorm(36,mean = 0,sd=3),nrow = 6,byrow = TRUE)
> r <- rast(nrows = 6,ncols = 6,xmin = 0, xmax = 6,ymin = 0, ymax = 6,
  crs = "local")
> values(r) <- vals # as.vector(t(vals))
# 3 x 3 focal maximum example
> r_focal <- focal(r,w = matrix(1, nrow = 3, ncol = 3),fun = max)
```

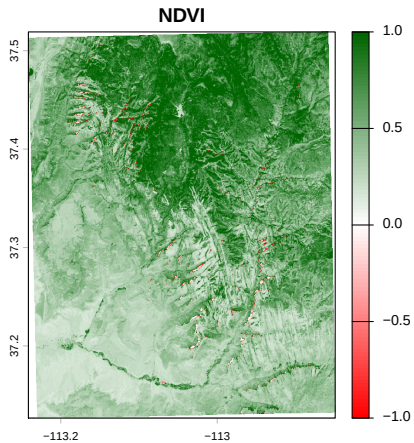


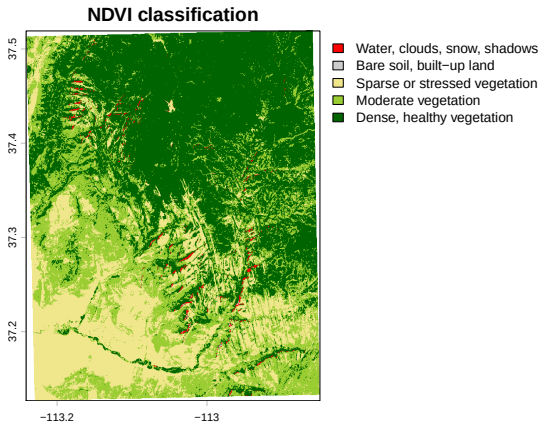
Raster algebra – Zonal operations with `zonal()`

- `zonal()` function applies some aggregation to multiple raster cells.
- Typically, second raster (categorical values) defines zonal filter ('zones').
- Cells defining the zonal filter do not have to be neighbors.
- The result is a summary table grouped by zone

NDVI summary by zone categorization example:

```
> terra::zonal(ndvi_rast, ndvi_class, fun = "mean")
|                      NDVI_class        NDVI
| 1  Water, clouds, snow, shadows -0.52244401
| 2      Bare soil, built-up land  0.03437564
| 3 Sparse or stressed vegetation  0.23252332
| 4           Moderate vegetation  0.39165562
| 5     Dense, healthy vegetation  0.75644240
```

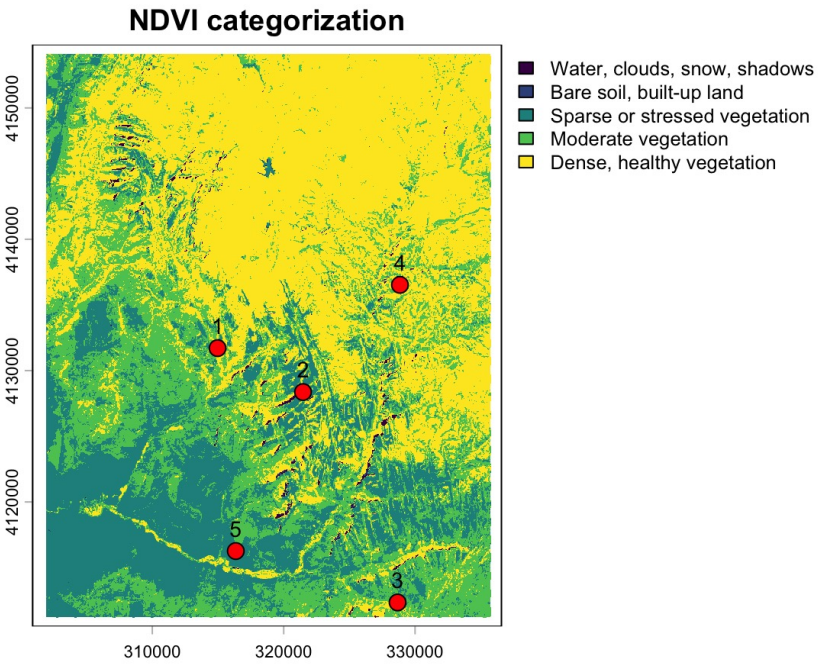


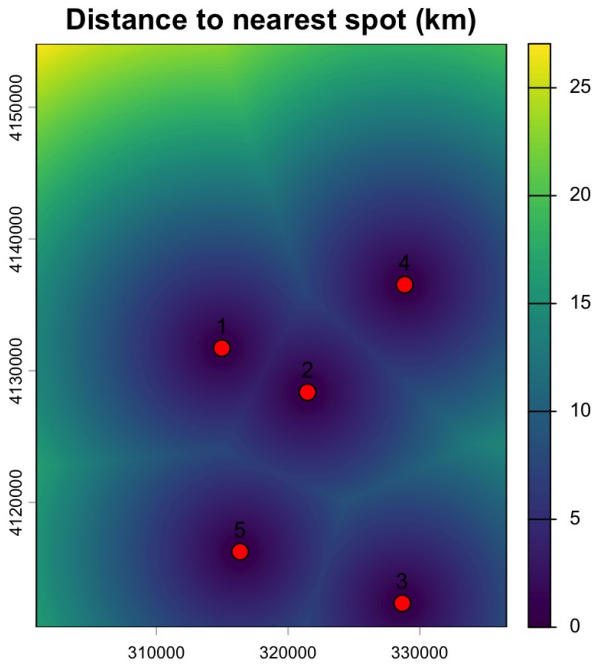


Raster algebra – Global (per raster) operations

For computation of distance and weight rasters (among other uses)

```
# project from WGS 84 to EPSG:32612 for simple distance calculation (m)
> ndvi_proj <- project(ndvi_class, "EPSG:32612")
# generate 5 random spots on a map; for example, emergency shelters
> spots <- spatSample(ndvi_proj,size = 5,method = "random",
>   as.points = TRUE,na.rm = TRUE,values = FALSE)
> dist_to_spot <- terra::distance(ndvi_proj,spots,method="geo",rasterize=FALSE)
# Convert aerial distances to kilometres, CRS units are metres
> dist_to_spot <- dist_to_spot / 1000
```



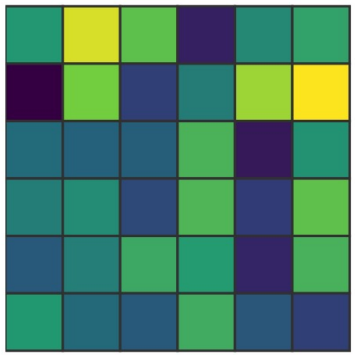


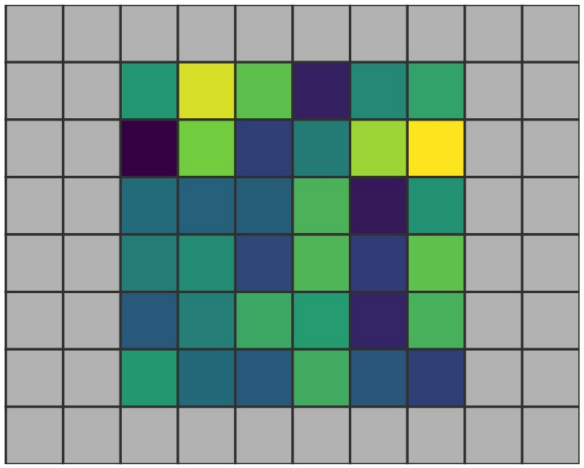
Rasters – data & geometry handling

For merging rasters (any map algebra operations): extent, origin, projection, and resolution have to match.

- **Extent:** Say, two rasters (images) differ in their extent. Math algebra can only be performed after extent harmonization.

```
# 6x6 raster
> r <- rast(nrows = 6,ncols = 6,xmin = 0, xmax = 6,ymin = 0, ymax = 6,crs = "local")
> values(r) <- rnorm(ncell(r))
# add 1 row above and below, and 2 columns left and right (NA values assigned)
> r_ext <- extend(r, c(1, 2))
> r + r_ext
| ## Error : [+] extents do not match
# change the extent of r to match r_ext
> r_new = extend(r, r_ext)
> r_sum <- r_new + r_ext
```





Rasters – data & geometry handling

For merging rasters (any map algebra operations): extent, origin, projection, and resolution have to match.

- **Origin:** the cell closest to the (0, 0) coordinates - top left cell in `terra` package.
  `terra::origin()` can be used to retrieve or set/change origin.

- **Projection:** With different or missing CRS, the projection must be set/harmonized before map algebra operations.
  `terra::crs()` get or set CRS,
  `terra::project()` for reprojections.

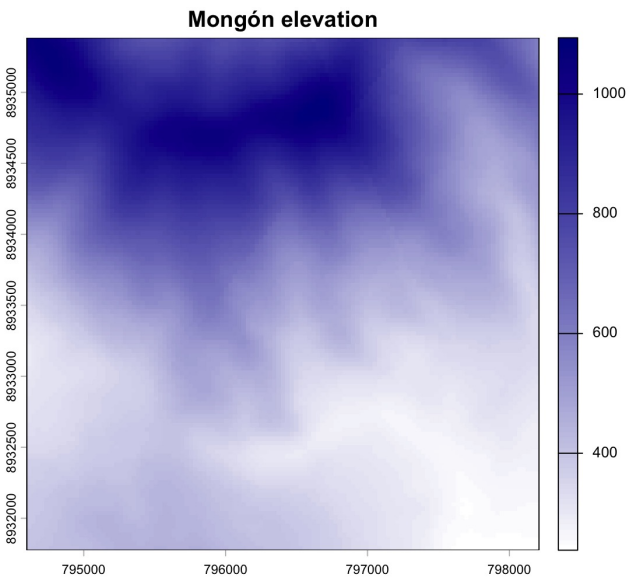
Rasters – data & geometry handling

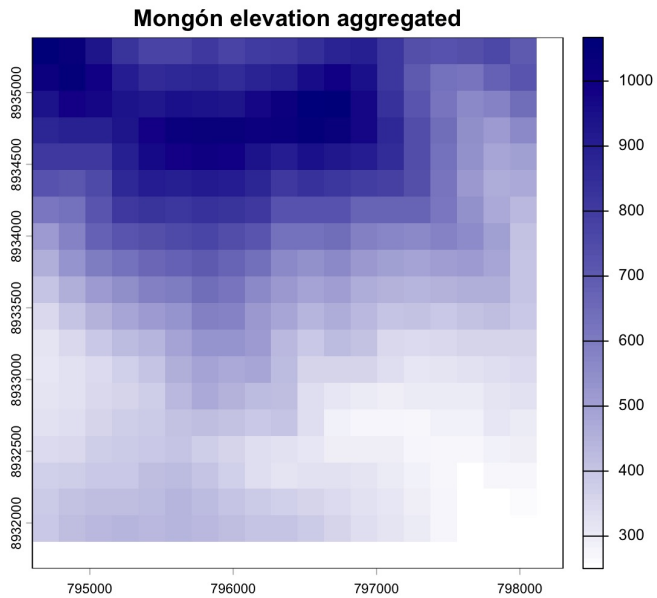
For merging rasters (any map algebra operations): extent, origin, projection, and resolution have to match.

- **Resolution** can be amended (harmonized) as follows:
  - `terra::aggregate()` decreases resolution by a given factor. Values of newly created cells are aggregated.
  - `terra::disagg()` increases resolution by a factor. Values in newly created cells are either simply repeated `(method = "near")` or interpolated, e.g.: `(method = "bilinear")`
  - `terra::resample()` is the most general operation, can be used on two or more rasters with different resolutions and origins. Takes the values of the original raster(s) and recalculates new values for a target raster with a given (different) resolution and origin. Reprojection – changing CRS – is a special case of resampling.

Aggregation example

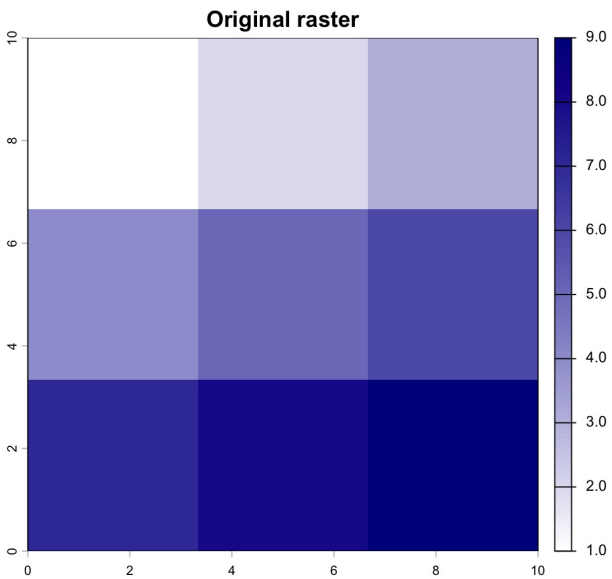
```
# Digital elevation model (DEM) of the Mongon study area.
> mongon <- rast(system.file("raster/dem.tif", package = "spDataLarge"))
> mongon_agg = aggregate(mongon, fact = 6, fun = mean)
```

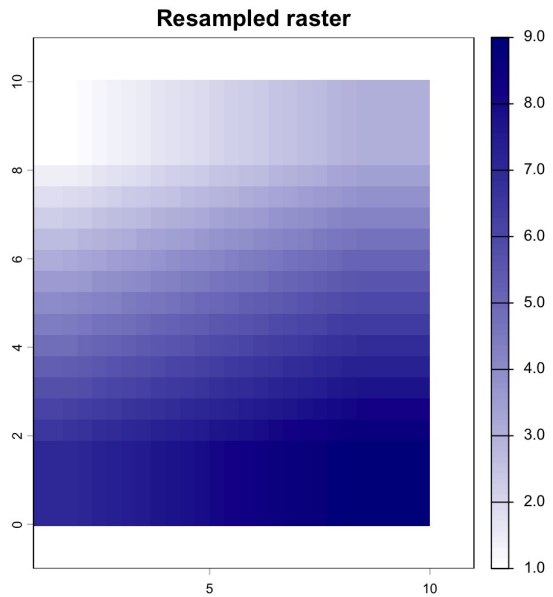




Resampling example

```
> r <- rast(nrows=3, ncols=3, xmin=0, xmax=10, ymin=0, ymax=10)
> values(r) <- 1:ncell(r)
> s <- rast(nrows=25, ncols=30, xmin=1, xmax=11, ymin=-1, ymax=11)
# note the different origin, extent and resolution of s
> x <- resample(r, s, method="bilinear")
> plot(r, col = pal_white_blue,main = "Original raster")
> plot(x, col = pal_white_blue,main = "Resampled raster")
```





Note: a closely related topic – **spatial interpolation** – discussed in Block 2.

Rasters – reading and writing data in `terra`

Table: Common spatial data formats for `SpatRaster` and `SpatVector` objects.

Format	**Data type**	**Main usage and features**
GeoTIFF `.tif .tiff`	Raster	Standard raster exchange; CRS, bands/layers, compression (not ideal for complex multid. data)
NetCDF `.nc`	Multidimensional raster	Time, depth, variables, climate data cubes May require choosing a variable as 1st step
GeoPackage `.gpkg`	Vector	Modern single-file vector format; multiple layers Not for `SpatRaster` objects
Shapefile GeoJSON	Vector Vector	Widely supported legacy format Web-friendly and human-readable
`.rds` / `.Rdata`	`R` objects	Convenient for `R`-only workflows

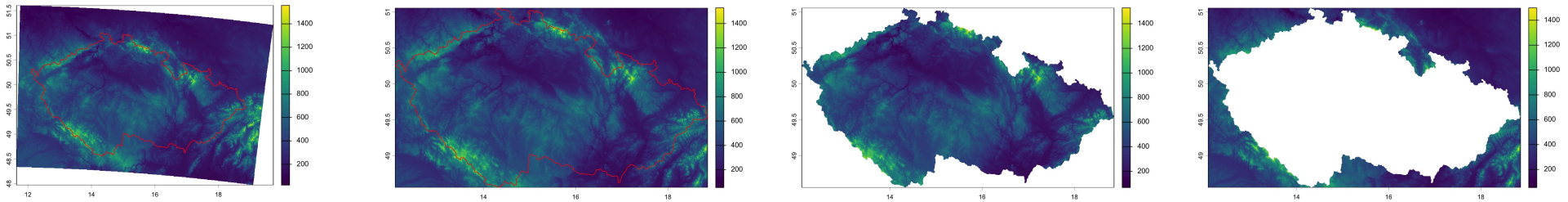
`r <- rast("myRaster.tif")` reads GeoTiff file as a raster

`writeRaster(r,"targetFile.tif",overwrite=TRUE)` save as GeoTiff

Raster-vector interactions

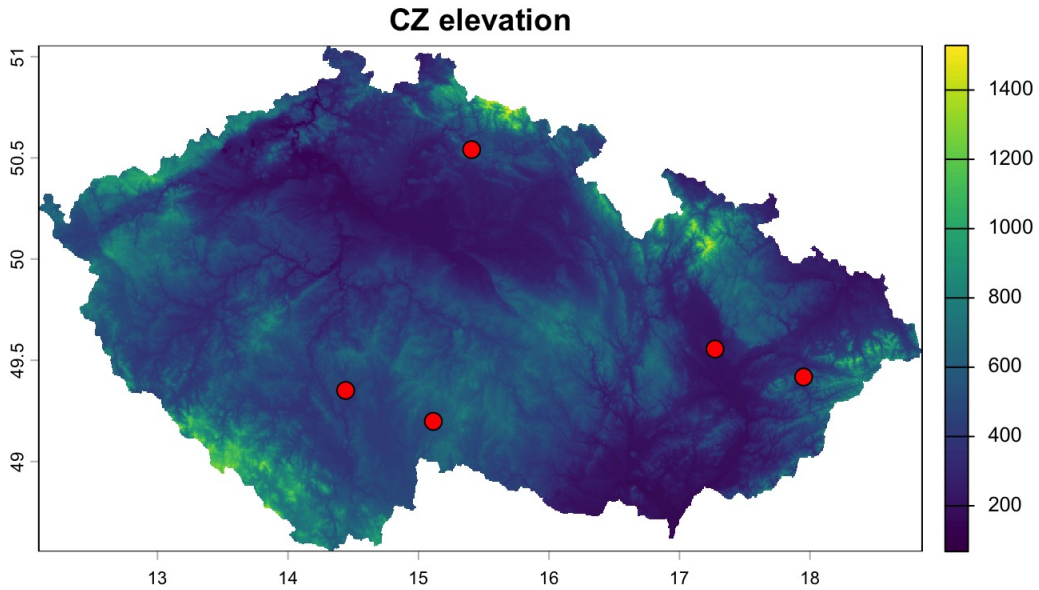
- `crop(x,y, ...)` reduces the rectangular extent of the `x` object to the extent of the `y` object (vector or raster).
- `mask(x,y, inverse = FALSE, ...)` sets the values of `x` that are beyond the extent of `y` to `NA`.
- The two functions are often used together.

CZ elevation raster, CZ state border (red), `crop()` and `mask()`.



Raster data extraction (points)

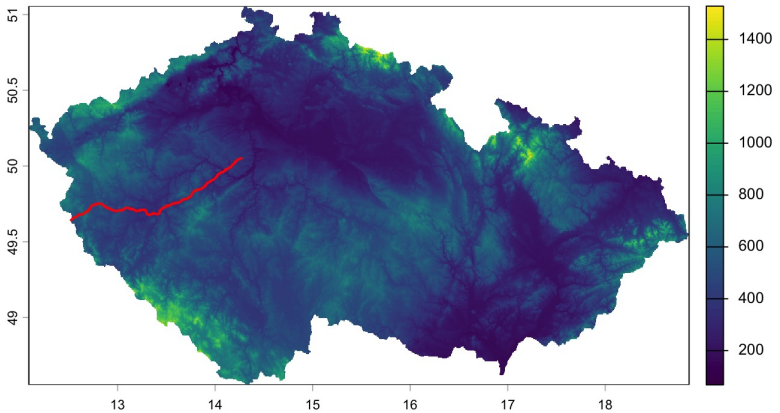
```
> CZspots <- spatSample(masked,size = 5,method = "random",
>  as.points = TRUE,na.rm = TRUE,values = FALSE)
> plot(masked,main = "CZ elevation")
> points(CZspots,pch = 21,bg = "red",col = "black",cex = 1.5)
> elevation <- terra::extract(masked, CZspots)
> elevation
|   ID   Band_1
| 1  1 422.0603
| 2  2 180.9770
| 3  3 795.9077
| 4  4 497.2205
| 5  5 471.8945
```



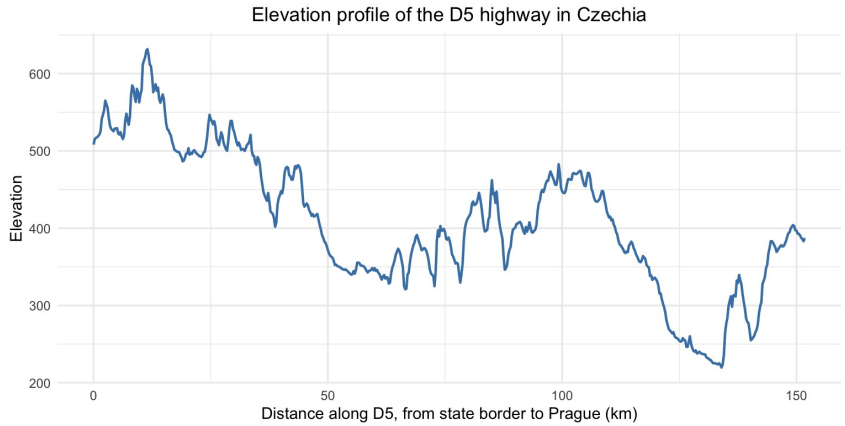
Raster data extraction (lines)

```
# D5 preparation:
# .. download from OSM, simplify, MULTILINESTRING > LINESTRING
# .. create points on line 250m apart to extract elevation
# Extract raster values
> D5_elev <- terra::extract(
>   masked,
>   terra::vect(D5_points_raster_crs))
```

CZ elevation raster with D5 highway (red);



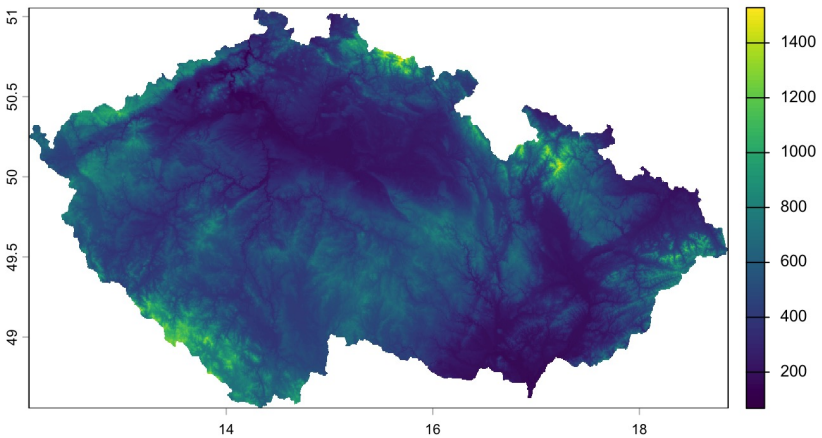
D5 elevation profile.

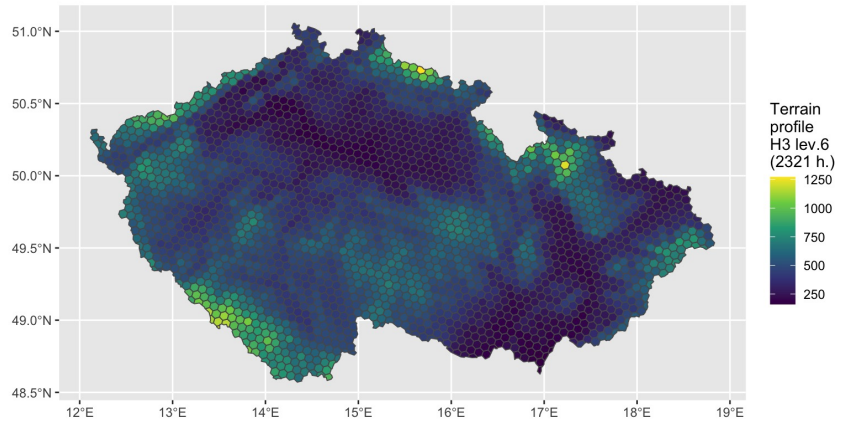


Raster data extraction (polygons)

```
# Retrieve H3 level 6 hexagons for Czechia
> hex_ids <- h3jsr::polygon_to_cells(CZ_buff, res = 6)
> hex_sf <- h3jsr::cell_to_polygon(hex_ids, simple = F)
> hex_sf <- hex_sf[st_intersects(hex_sf, CZbound, sparse = F), ]
> hex_sf <- st_intersection(hex_sf, CZbound)
#
> hex_sf$elevation <- exactextractr::exact_extract( # faster than terra::extract()
>   x = masked, # source, masked raster with CZ profile
>   y = hex_sf, # target, sf polygons
>   fun = "mean", # calculation of average built-up - can be used for air pollution, etc.
>   weights = "area", coverage_area = TRUE,
>   progress = FALSE) # hide console progress visualization
```

CZ elevation raster; extraction to H3 (Uber) hexagonal grid (lev. 6)





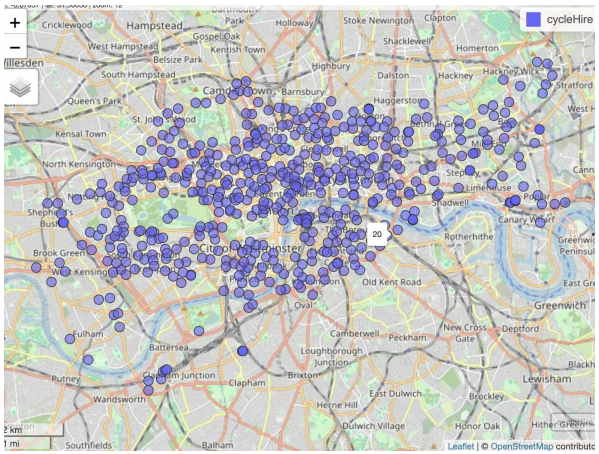
Rasterization (points-to-raster example)

Rasterization: conversion of vector objects (sf-type points, or other) into a raster-form representation.

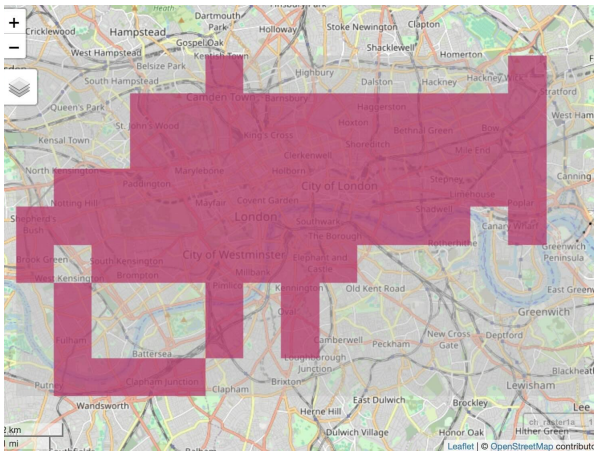
```
> cycleHire <- spData::cycle_hire_osm
> cycleHire_projected <- st_transform(cycleHire, "EPSG:27700")
> raster_template <- rast(ext(cycleHire_projected), resolution = 1000,
>                         crs = crs(cycleHire_projected))
# presence/absence
> ch_raster1 <- rasterize(cycleHire_projected, raster_template)
# different cycle hire locations have different numbers of bicycles
# -> calculate total capacity in each grid cell (capacity variable)
> ch_raster2 <- rasterize(cycleHire_projected, raster_template,
>                         field = "capacity", fun = sum, na.rm = TRUE)
```

Bicycle hire sites in London – rasterization example

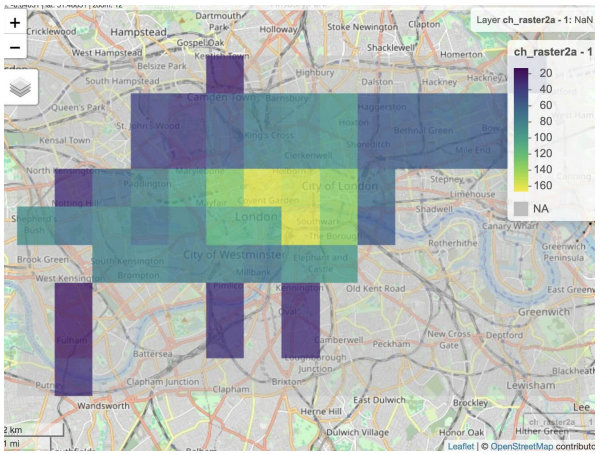
Points



Presence / absence



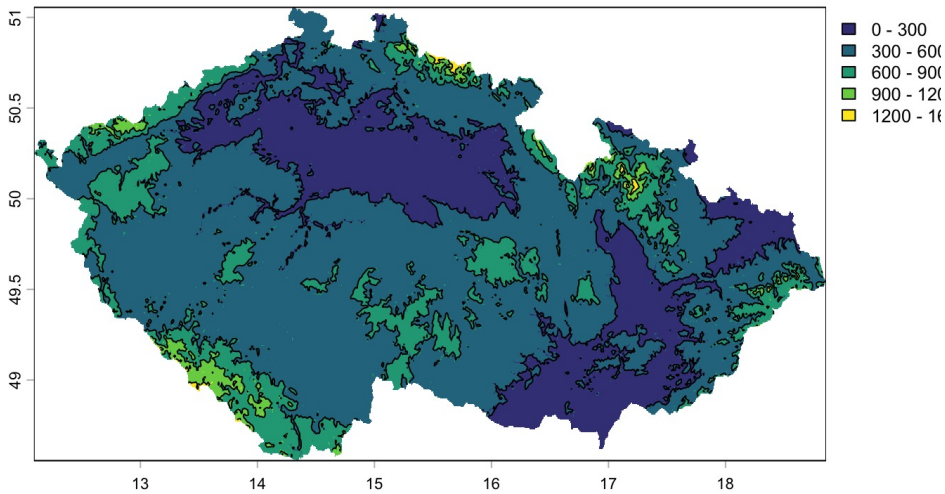
Hire capacity



Spatial vectorization (elevation-to-contour example)

Spatial vectorization: counterpart of rasterization. Conversion of spatially continuous raster data to vector-type data (points, lines, polygons).

```
> as.contour(masked) |> st_as_sf()
| Simple feature collection with 7 features and 1 field
| Geometry type: MULTILINESTRING
| Dimension:     XY
| Bounding box:  xmin: 12.10985 ymin: 48.5723 xmax: 18.8475 ymax: 51.0405
| Geodetic CRS:  WGS 84
|   level                       geometry
| 1   200 MULTILINESTRING ((13.50001 ...
| 2   400 MULTILINESTRING ((12.57757 ...
| ...
# Plot, use different breaks for the contour lines
> br <- c(0, 300, 600,900,1200,1604)
> plot(masked, breaks=br)
> lines(as.contour(masked, levels=br))
```



Rasters – `R` lab

- Key raster operations
- Combining `sf` and raster data for analysis
- Remote sensing data: NDVI, NDWI
- Raster-based models (OLS, GLS)
- Rasterization (vector-to-raster data operations)